Machine learning

Neural networks/deep learning

Thomas D. Nielsen

Resources

The book *Neural Networks and Deep Learning* by Michael Nielsen:

`http://neuralnetworksanddeeplearning.com/`

The book *Probabilistic machine learning: An Introduction* by Kevin Murphy:

`https://probml.github.io/pml-book/book1.html`

Colah's blog by Christopher Olah:

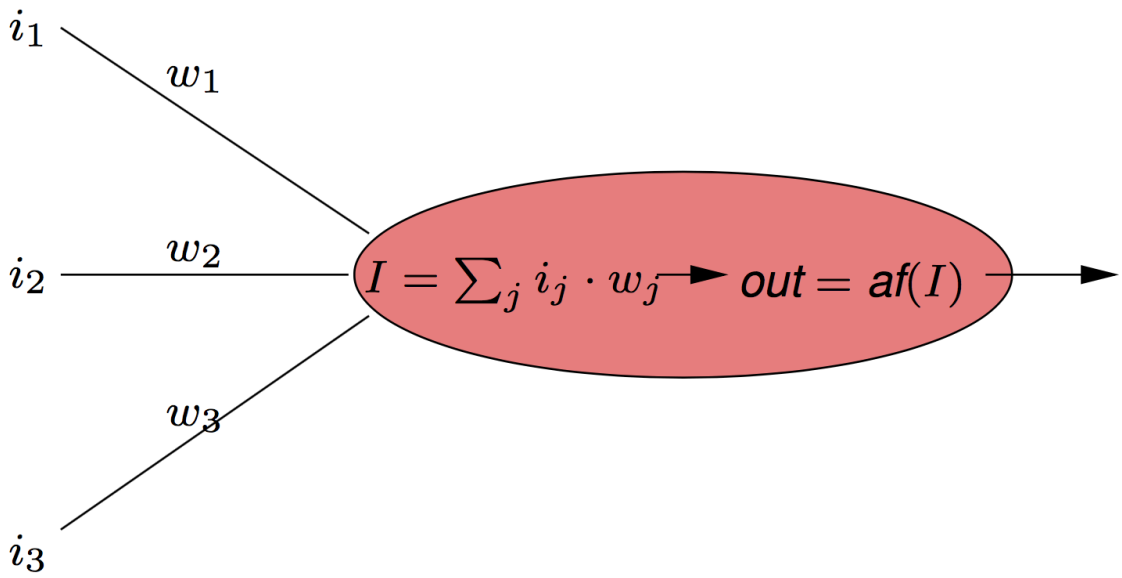
`http://colah.github.io/`

NB:

- Some slides partly based on Ian Goodfellow's lecture slides for the Deep Learning book
- Some figures prepared by Manfred Jaeger

Deep feed forward networks

Neural networks: basic building unit



Two step computation:

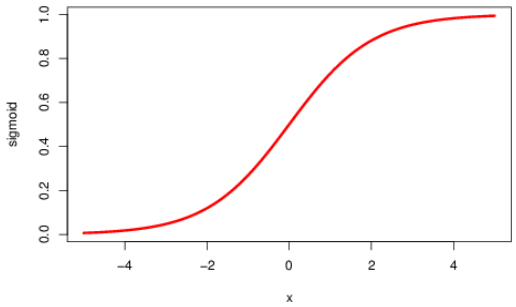
- Combine inputs as *weighted sum* (vector representation: $\mathbf{w}^\mathsf{T}\mathbf{i} = \sum_j i_j w_j$)
- Compute output by **activation function** of combined inputs

Common activation functions

Some common activation functions are:

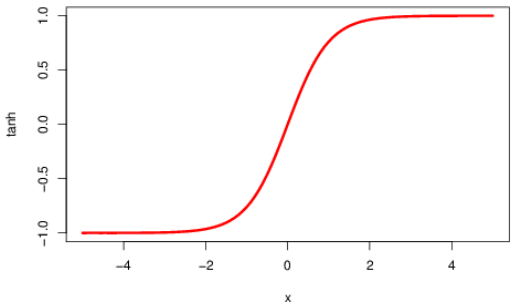
Sigmoid

$af(x) = 1/(1 + e^{-x})$



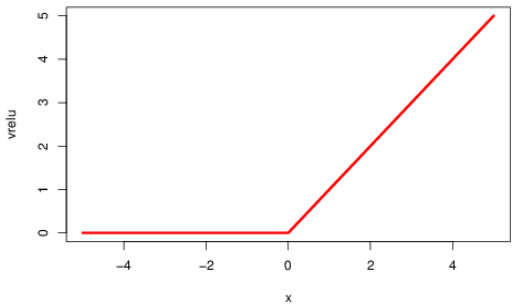
Hyperbolic tangent

$af(x) = \tanh(x)$



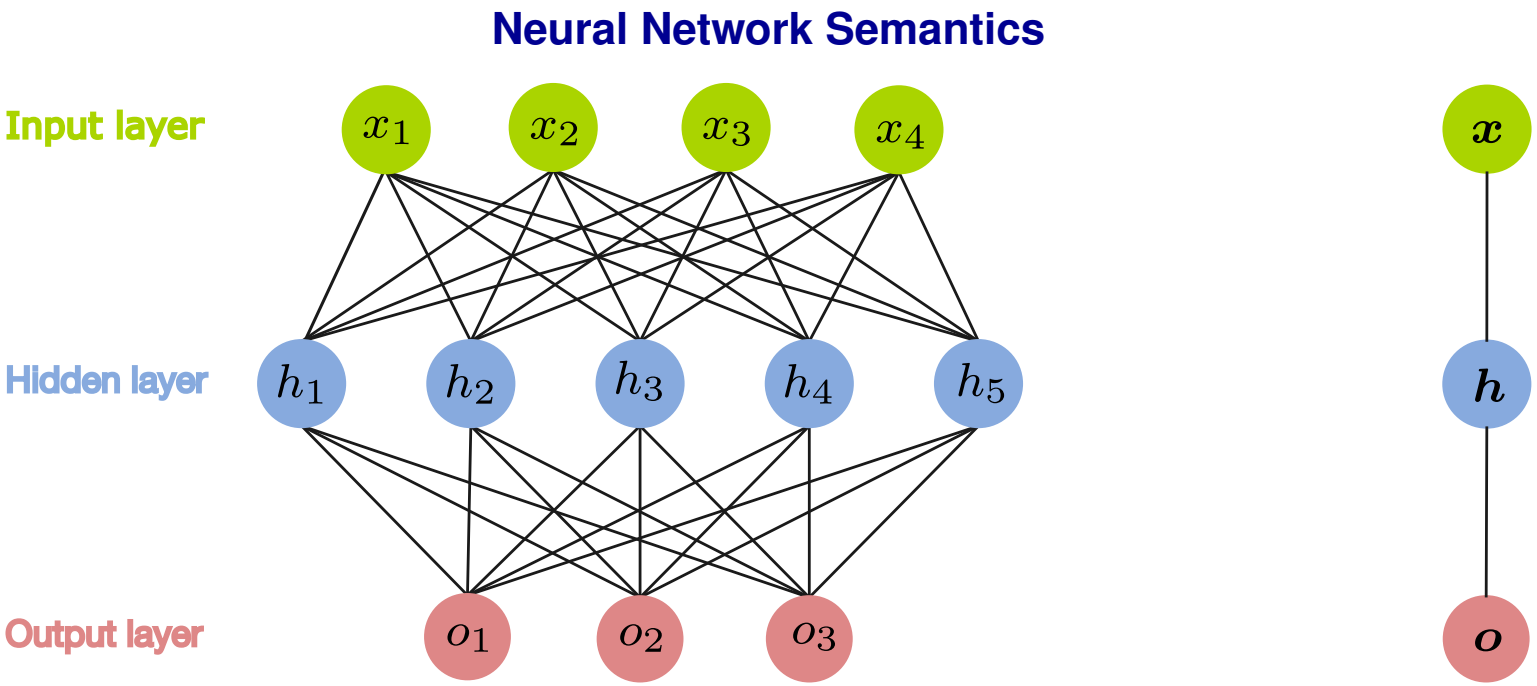
Relu

$af(x) = \max(0, x)$



- If activation function is sigmoid, i.e. $out = \sigma(\sum_j i_j \cdot w_j)$, we also talk of *squashed linear function*.
- For the output neuron also the **identity function** is used: $af(x) = id(x) = x$

Neural network semantics



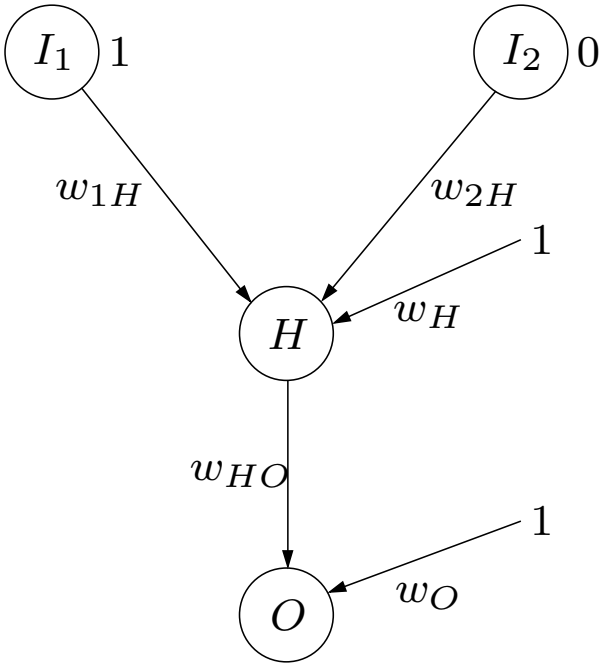
A neural network with n input and k output nodes defines a real-valued function on continuous input attributes:

$$f : \mathbb{R}^4 \rightarrow \mathbb{R}^3.$$

as a composite function:

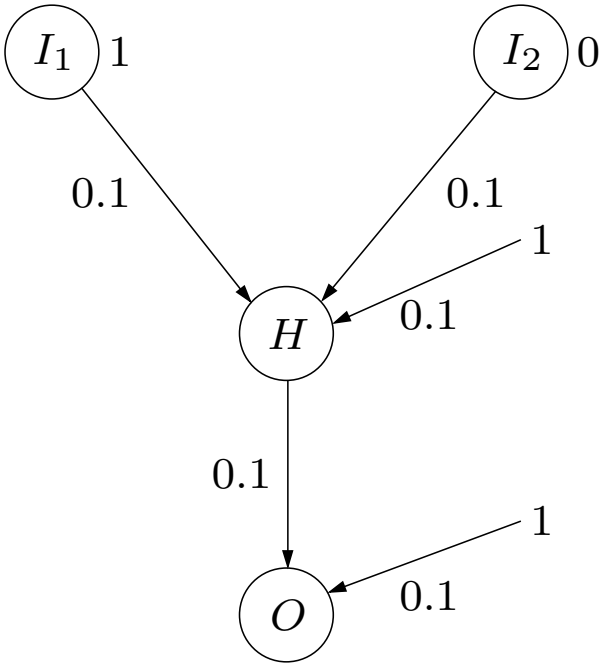
$$f(\mathbf{x}) = \mathbf{y}$$
$$\mathbf{y} = f_2(W_2^T \mathbf{h} + \mathbf{b}_2)$$
$$\mathbf{h} = f_1(W_1^T \mathbf{x} + \mathbf{b}_1)$$

Propagation in Neural Networks



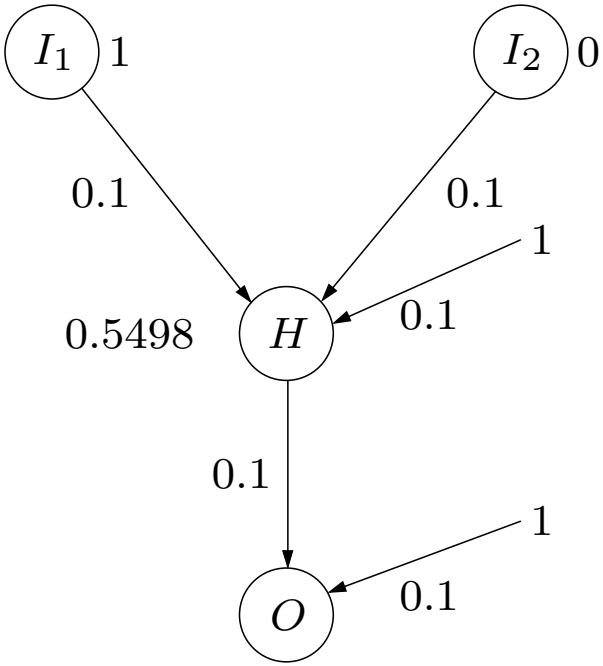
The input nodes are set to 1 and 0, respectively.

Propagation in Neural Networks



The input nodes are set to 1 and 0, respectively.

Propagation in Neural Networks

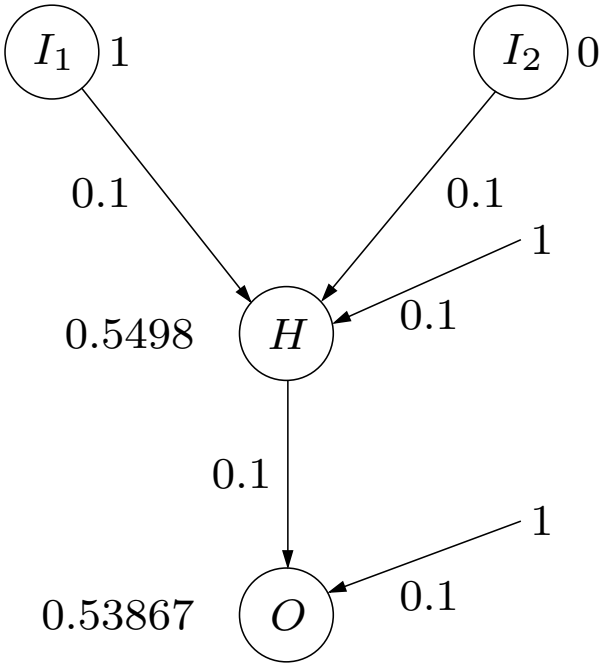


The output of of neuron H is:

$$o_H = \sigma(1 \cdot 0.1 + 0 \cdot 0.1 + 1 \cdot 0.1) = 0.5498.$$

The input nodes are set to 1 and 0, respectively.

Propagation in Neural Networks



The output of of neuron O is:

$$o_H = \sigma(0.5498 \cdot 0.1 + 1 \cdot 0.1) = 0.53867.$$

The input nodes are set to 1 and 0, respectively.

Learning: basics

The Task of Learning

Given:

- the structure and activation functions of the neural network
- the training examples

Input				Output			
X_1	X_2	$\ldots$	X_n	T_1	T_2	$\ldots$	T_m
$x_{1,1}$	$x_{2,1}$	$\ldots$	$x_{n,1}$	$t_{1,1}$	$t_{2,1}$	$\ldots$	$t_{m,1}$
$x_{1,2}$	$x_{2,2}$	$\ldots$	$x_{n,2}$	$t_{1,2}$	$t_{2,2}$	$\ldots$	$t_{m,2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$x_{1,N}$	$x_{2,N}$	$\ldots$	$x_{n,N}$	$t_{1,N}$	$t_{2,N}$	$\ldots$	$t_{m,N}$

- a *loss* function measuring the error between the target values $\mathbf{t}$ and the output values $\mathbf{o}$:

$$\mathit{loss}(\mathbf{t}, \mathbf{o})$$

Goal: Find the weights $\mathbf{w}$ that minimize the loss over the training examples:

$$\mathit{Loss} = \frac{1}{N} \sum_{i=1}^{N} \mathit{loss}(\mathbf{t}_i, \mathbf{o}_i(\mathbf{w}))$$

Error functions

Squared error

$$\mathit{loss}(\mathbf{t}, \mathbf{o}) = \sum_{i=1}^{m} (t_i - o_i)^2$$

- With a single output neuron, $m = 1$.
- Often used for regression problems.

Log-loss

$$\mathit{loss}(\mathbf{t}, \mathbf{o}) = -\log p(\mathbf{t} \mid \mathbf{x}),$$

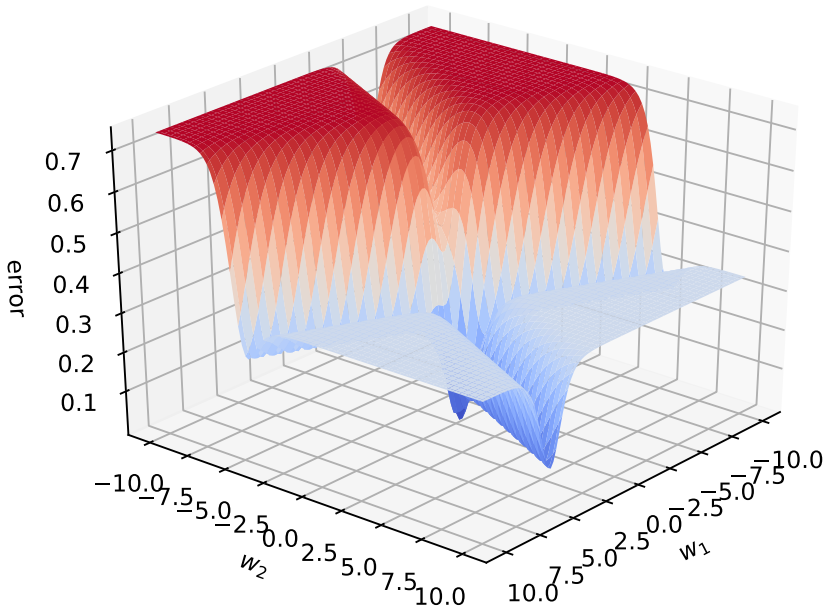
- Specific form of loss depends on p.
- For $p(\mathbf{t} \mid \mathbf{x}) \sim \mathcal{N}(f(\mathbf{x} \mid \mathbf{w}), \mathbf{I})$, loss is similar to squared error.

Cross-entropy

$$\mathit{loss}(\mathbf{t}, \mathbf{o}) = -\sum_{i=1}^{m} t_i \cdot \log(o_i)$$

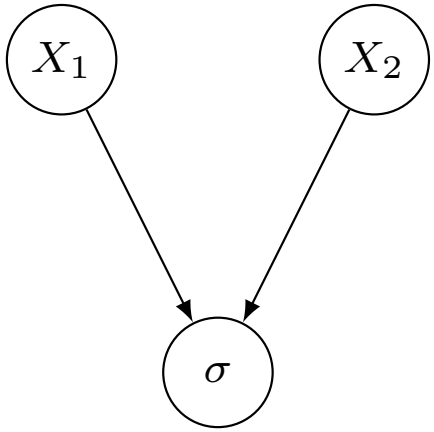
- Outputs should be in the interval $[0, 1]$ (interpreted as probabilities of the m classes).

Error function: example



Error surface with a squared error function

Network structure

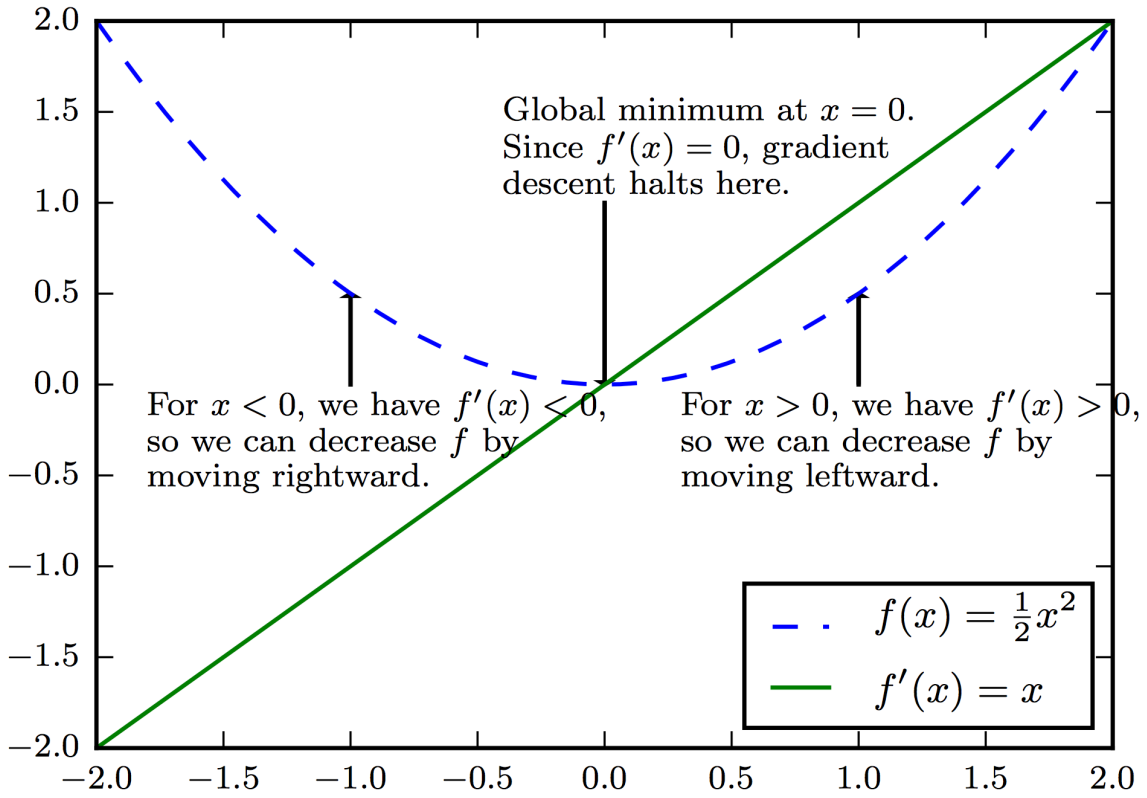


Data

X_1	X_2	O
1	2	0.7
2	−1	0.5

Learning: gradient-based

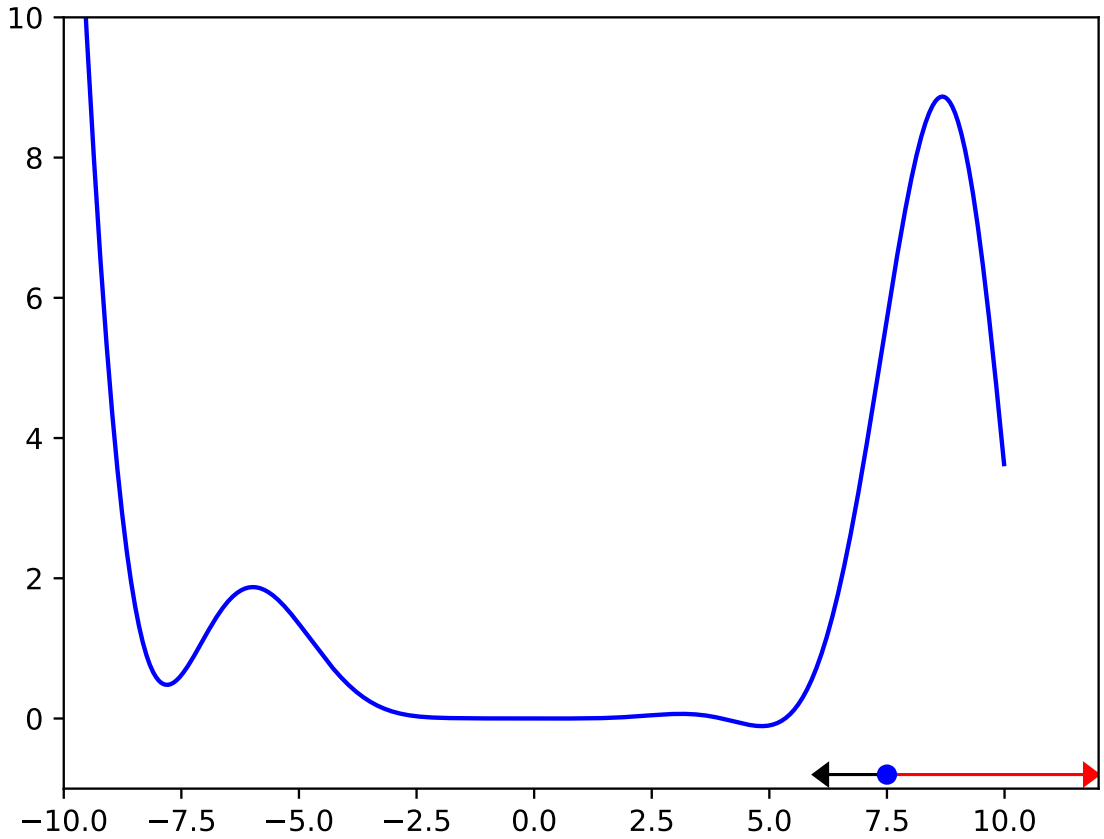
Learning usually involve gradient-based algorithms minimizing a cost function:



Simple gradient descent:

$$\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$$

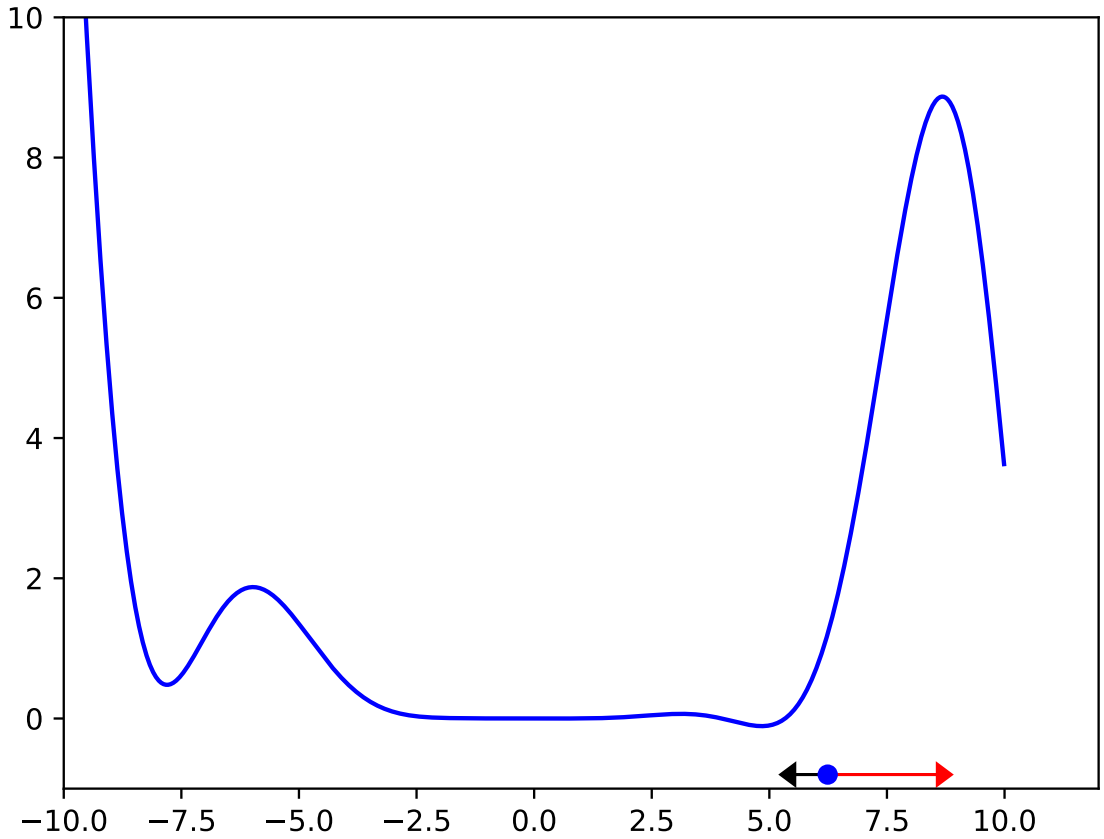
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

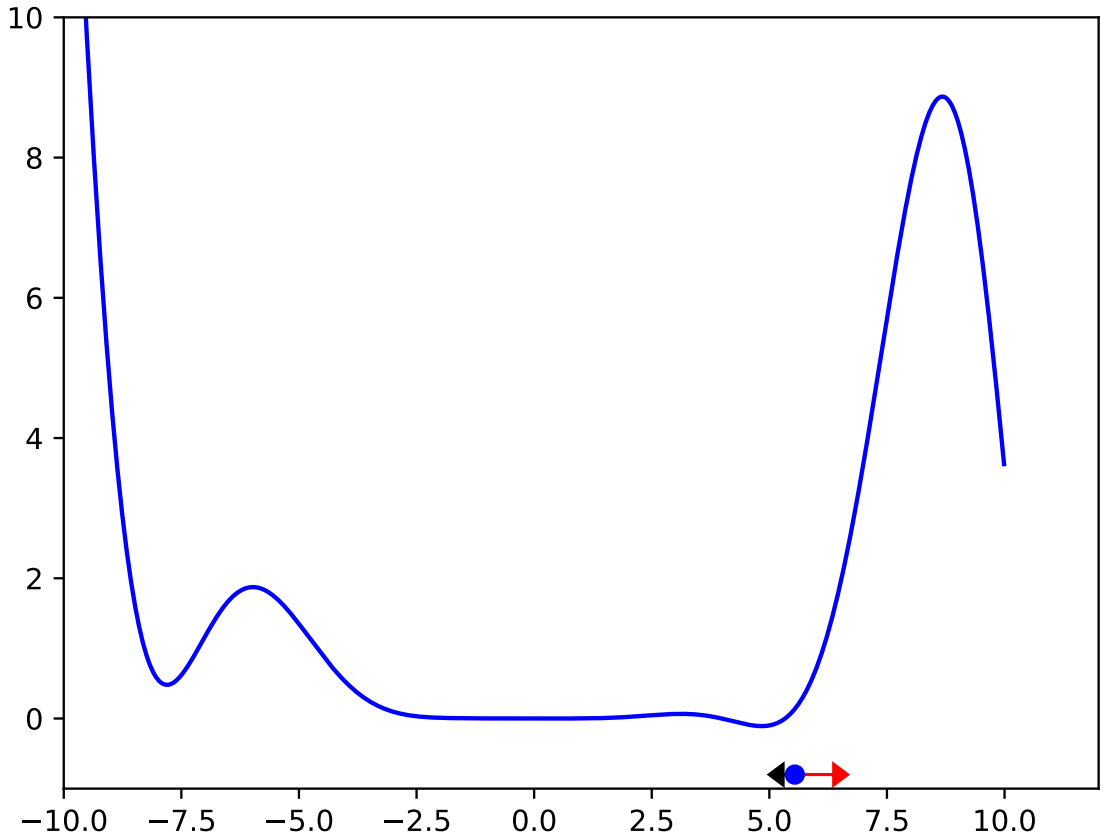
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

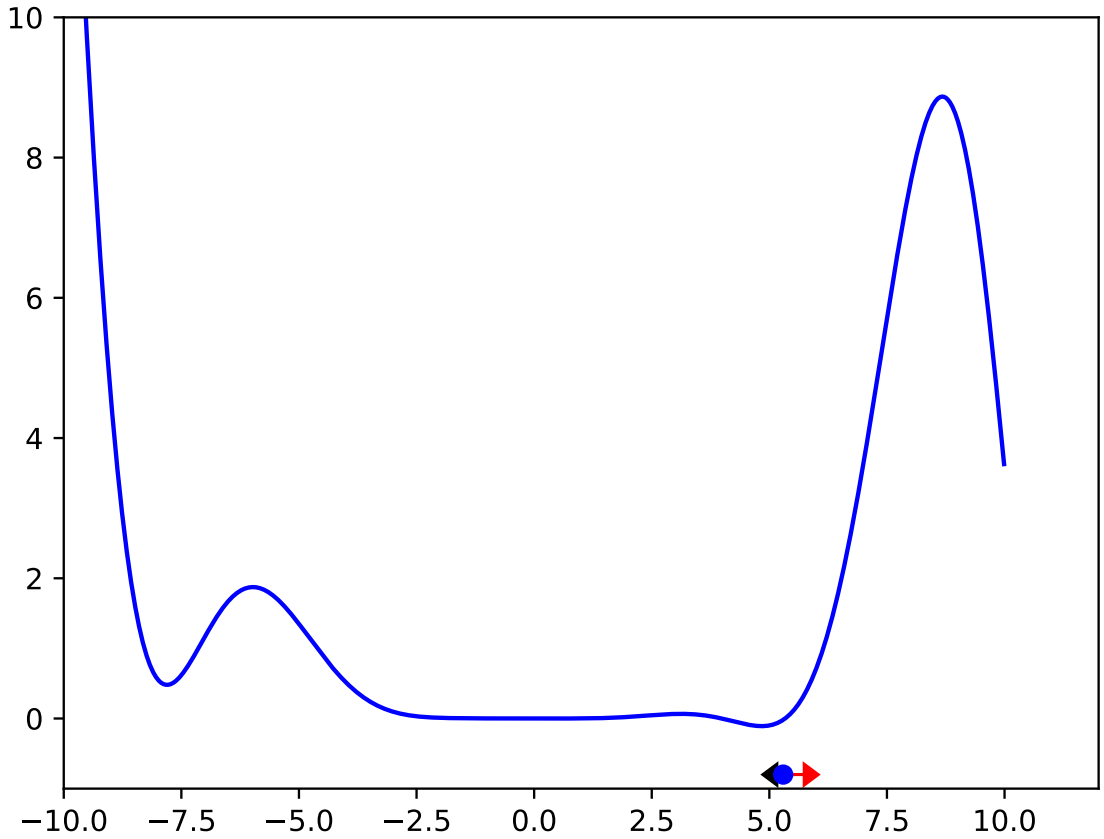
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

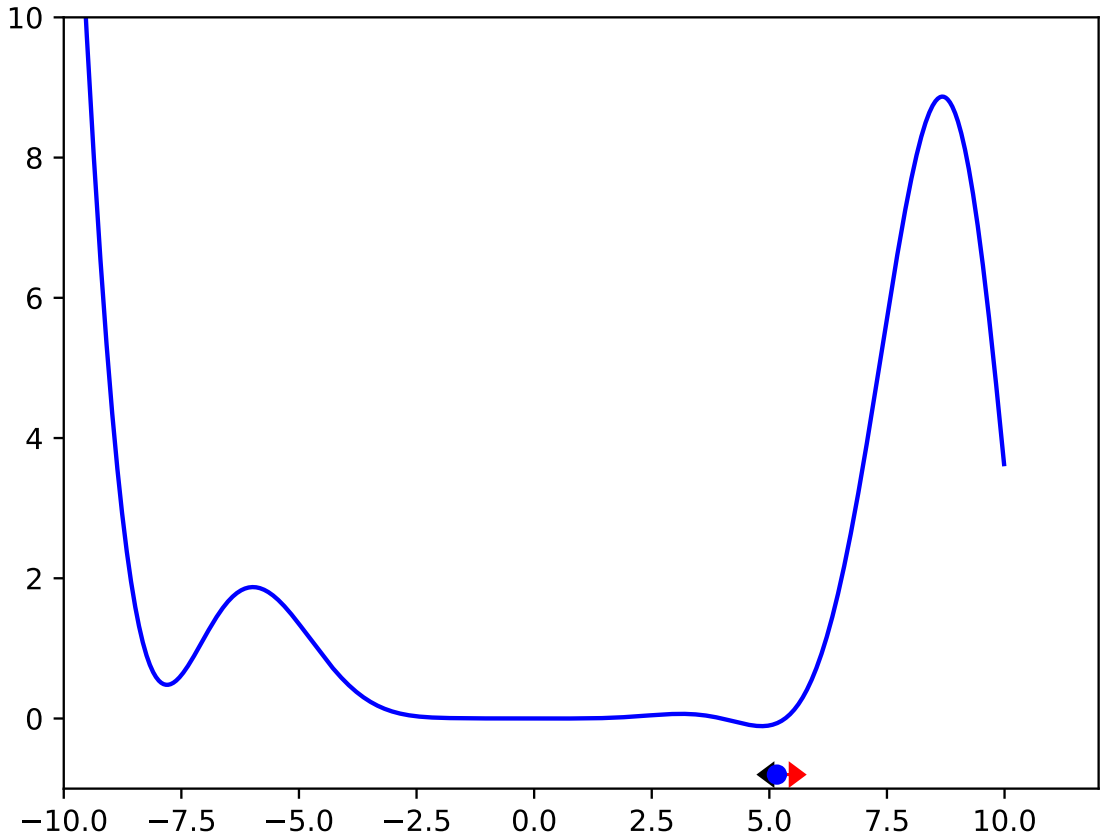
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

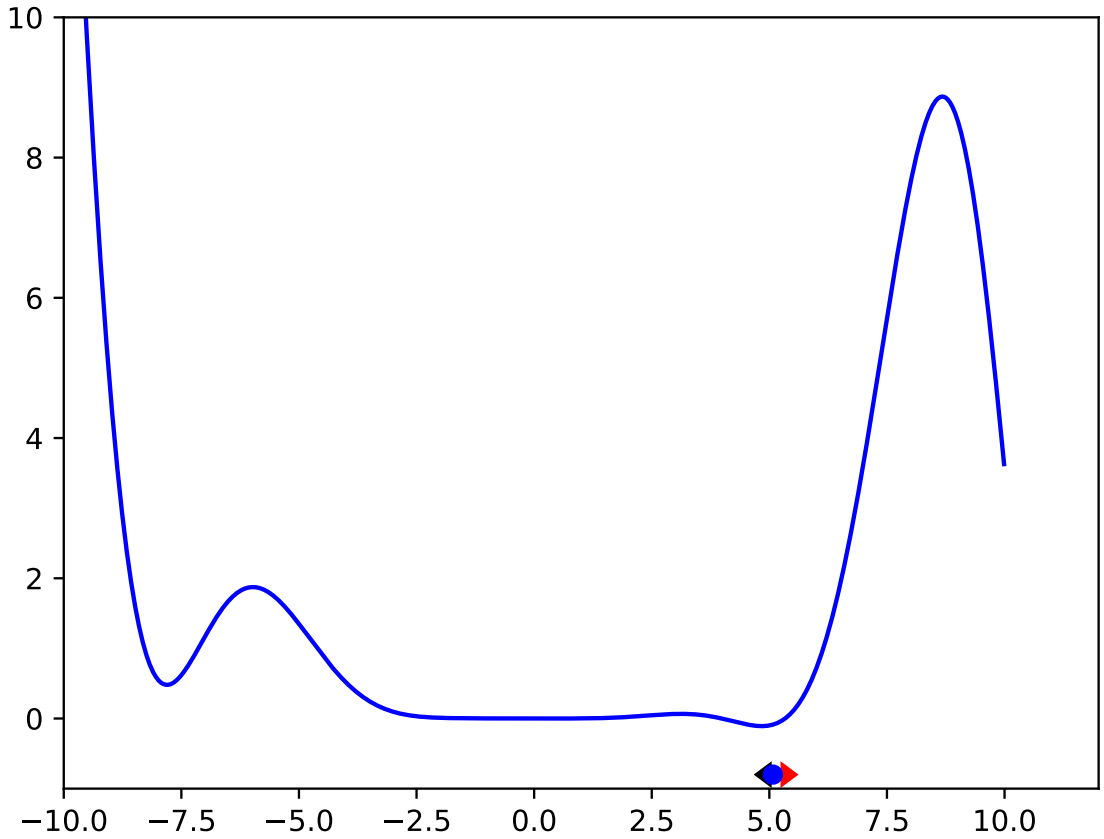
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

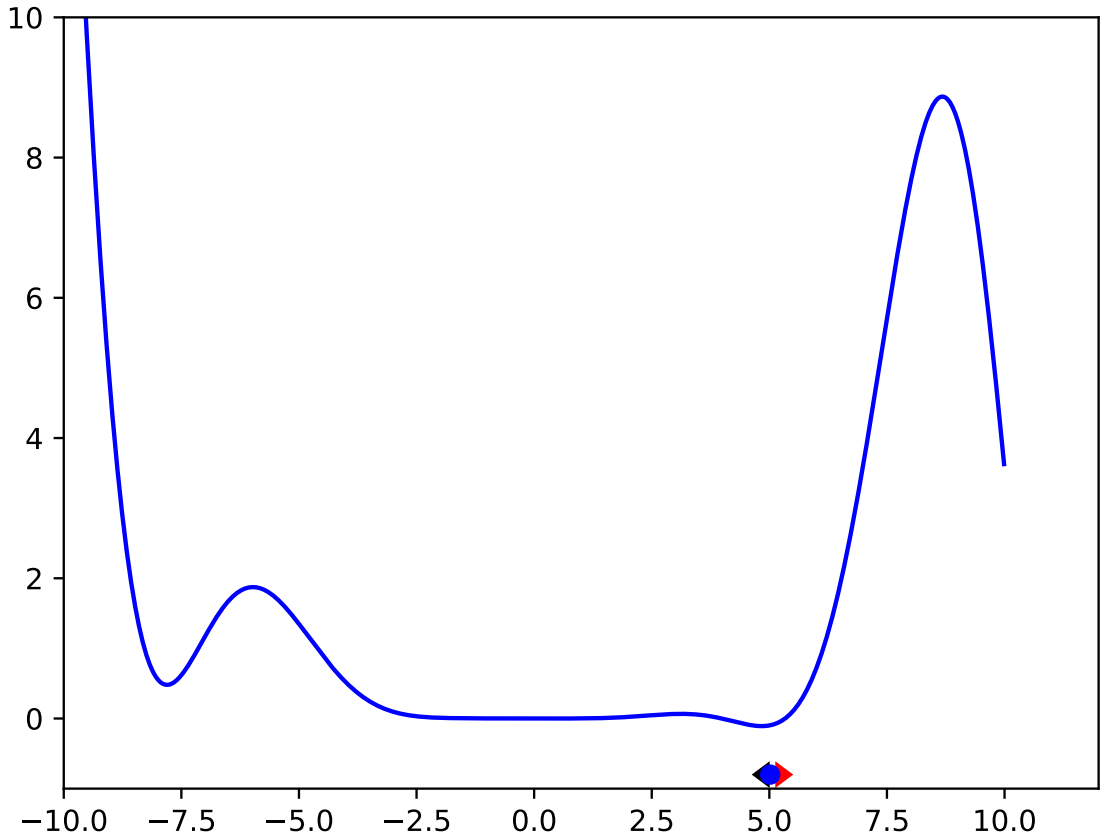
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

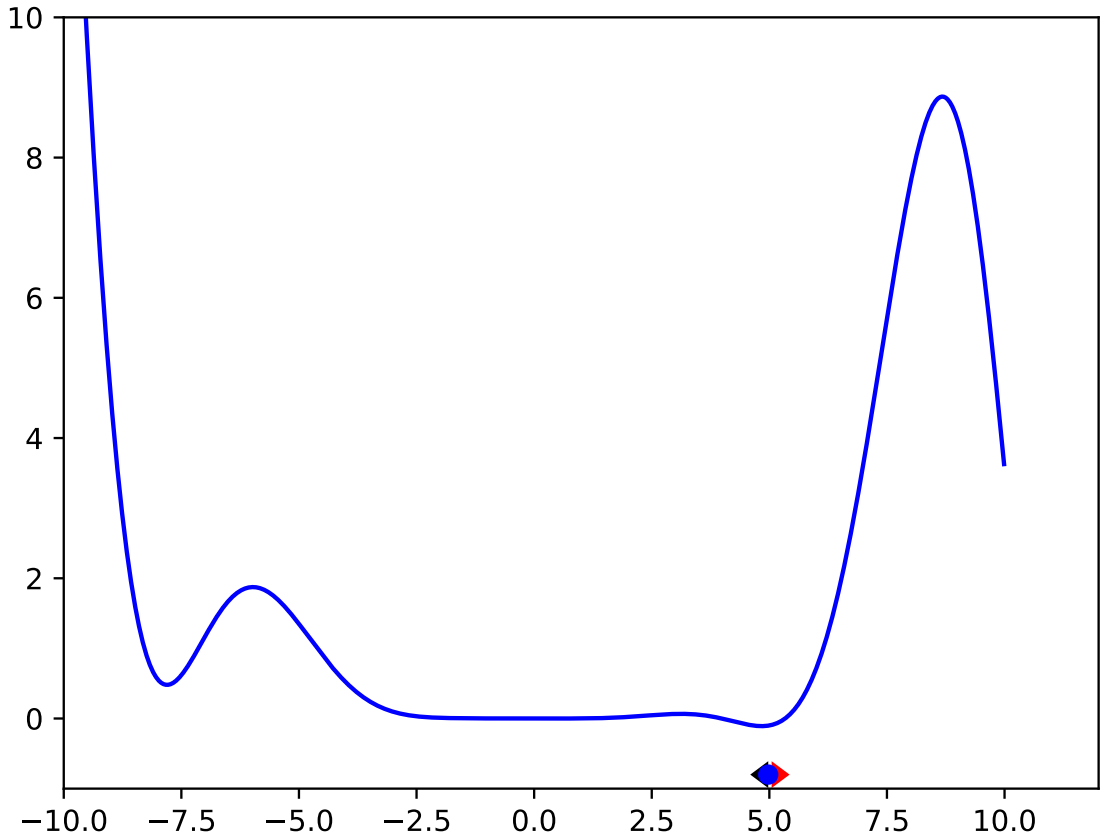
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

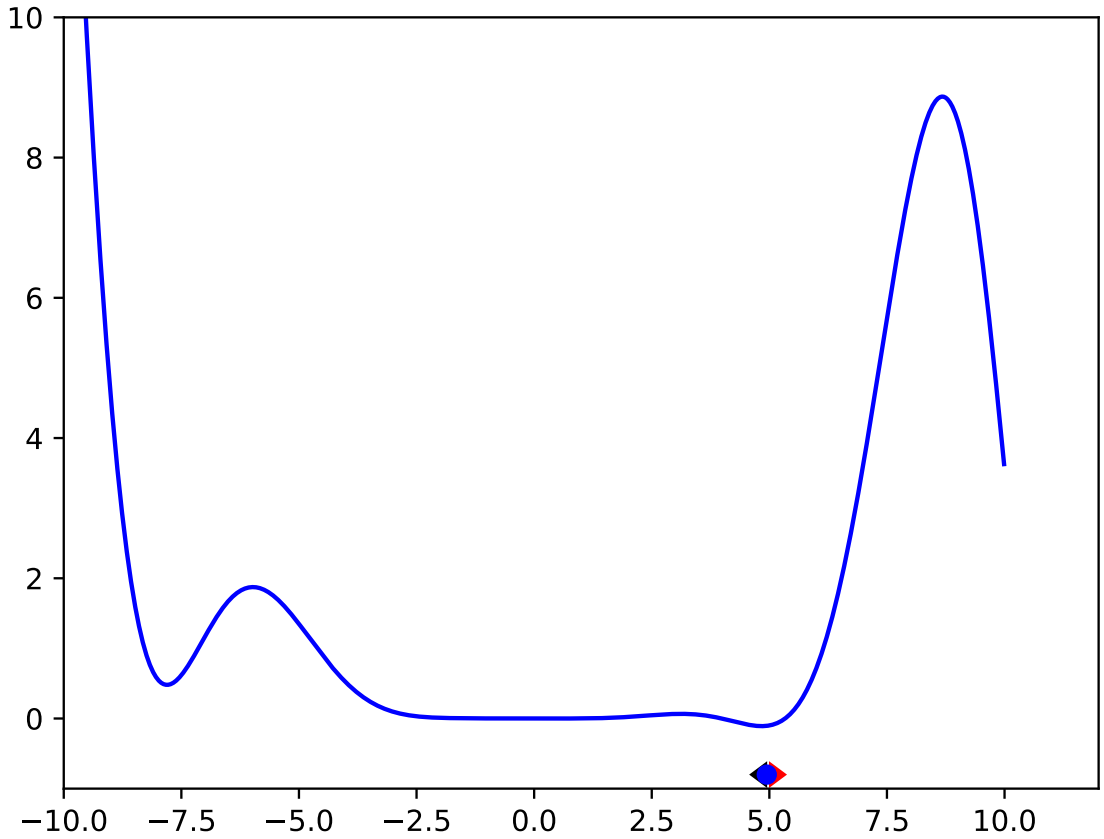
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

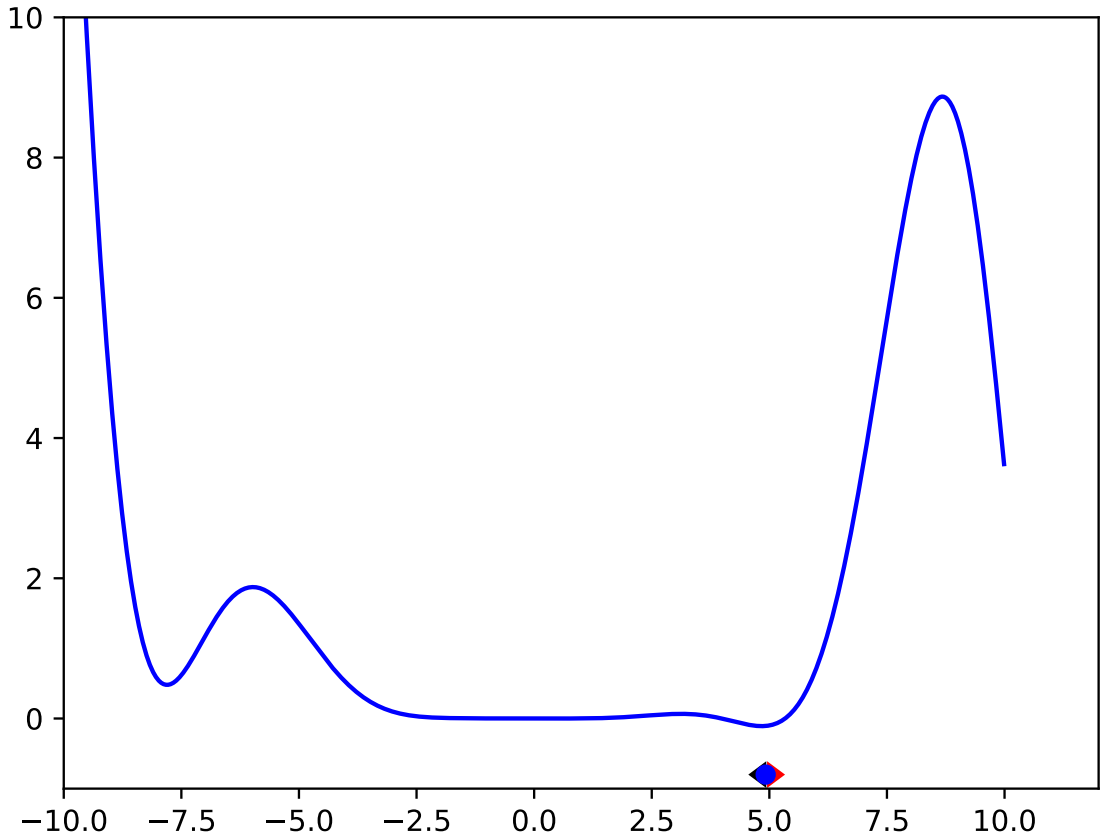
Gradient descent: an example



Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

Gradient descent: an example



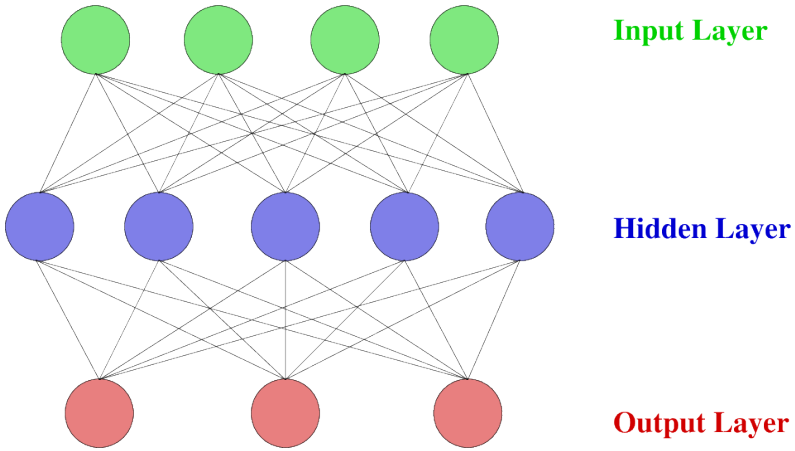
Gradient descent:

- Pick a random starting point $\mathbf{w}$
- Calculate the gradient/derivative $\nabla \mathit{Loss}(\mathbf{w})$
- Move in the opposite direction of the gradient: $\mathbf{w}' = \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w})$
- Until convergence . . . (convergence depends on learning rate η)

Back propagation

Back propagation

Training examples provide target values for only network outputs, so no target values are directly available for indicating the error of the hidden units' values.



$\rightsquigarrow$ compute local error terms for hidden nodes and adjust weights (through partial derivatives) using local terms.

Back propagation

An algorithm that computes derivaties using the chain rule.

The scalar case

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

With multiple component functions

$$\frac{\partial f(g_1(x), g_2(x))}{\partial x} = \frac{\partial f(g_1(x), g_2(x))}{\partial g_1(x)} \frac{\partial g_1(x)}{\partial x} + \frac{\partial f(g_1(x), g_2(x))}{\partial g_2(x)} \frac{\partial g_2(x)}{\partial x}$$

Example

For the function $f(x) = \sigma(x) \cdot x^2$ we have

$$\begin{aligned} \frac{\partial f(x)}{\partial x} &= \frac{\partial f(x)}{\partial \sigma(x)} \frac{\partial \sigma(x)}{\partial x} + \frac{\partial f(x)}{\partial x^2} \frac{\partial x^2}{\partial x} \\ &= x^2 \cdot \sigma'(x) + 2\sigma(x) \cdot x \end{aligned}$$

Back propagation

An algorithm that computes derivaties using the chain rule.

The scalar case

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

With multiple component functions

$$\frac{\partial f(g_1(x), g_2(x))}{\partial x} = \frac{\partial f(g_1(x), g_2(x))}{\partial g_1(x)} \frac{\partial g_1(x)}{\partial x} + \frac{\partial f(g_1(x), g_2(x))}{\partial g_2(x)} \frac{\partial g_2(x)}{\partial x}$$

A note on notation: Vector valued functions

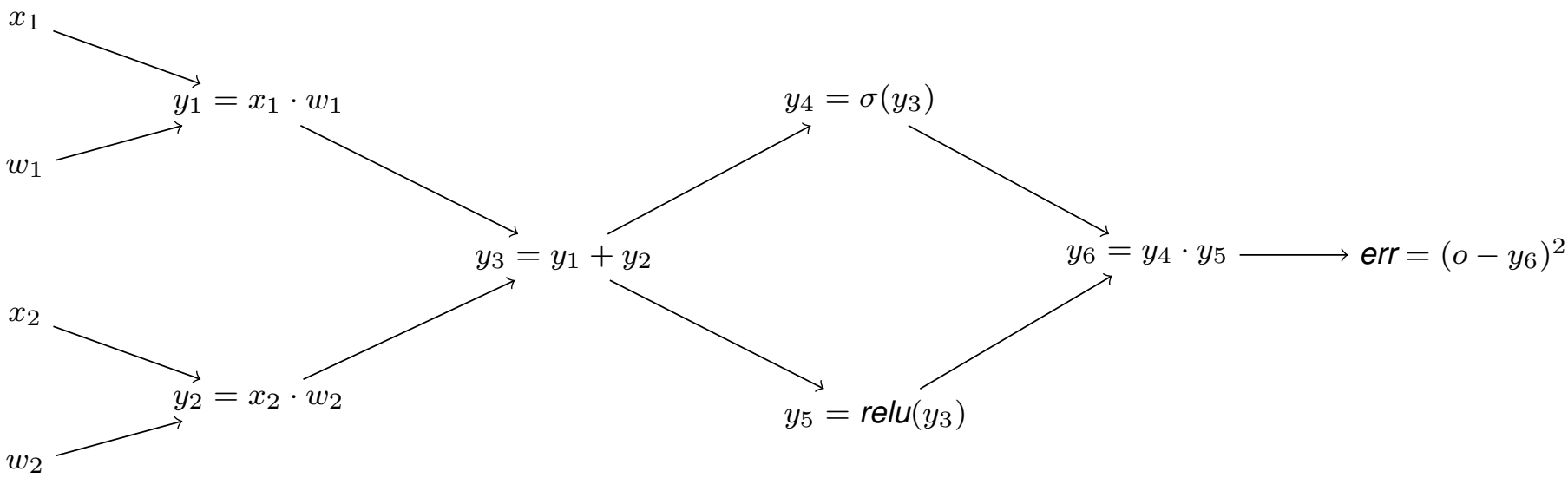
For $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{y} \in \mathbb{R}^n$, if $\mathbf{y} = g(\mathbf{x})$ and $\mathbf{z} = f(g(\mathbf{x})) = f(\mathbf{y})$, then

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$$

In vector notation: $\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^{\mathsf{T}} \nabla_{\mathbf{y}} z$, where $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ is the $n \times m$ Jacobian matrix.

Example

Computational graph

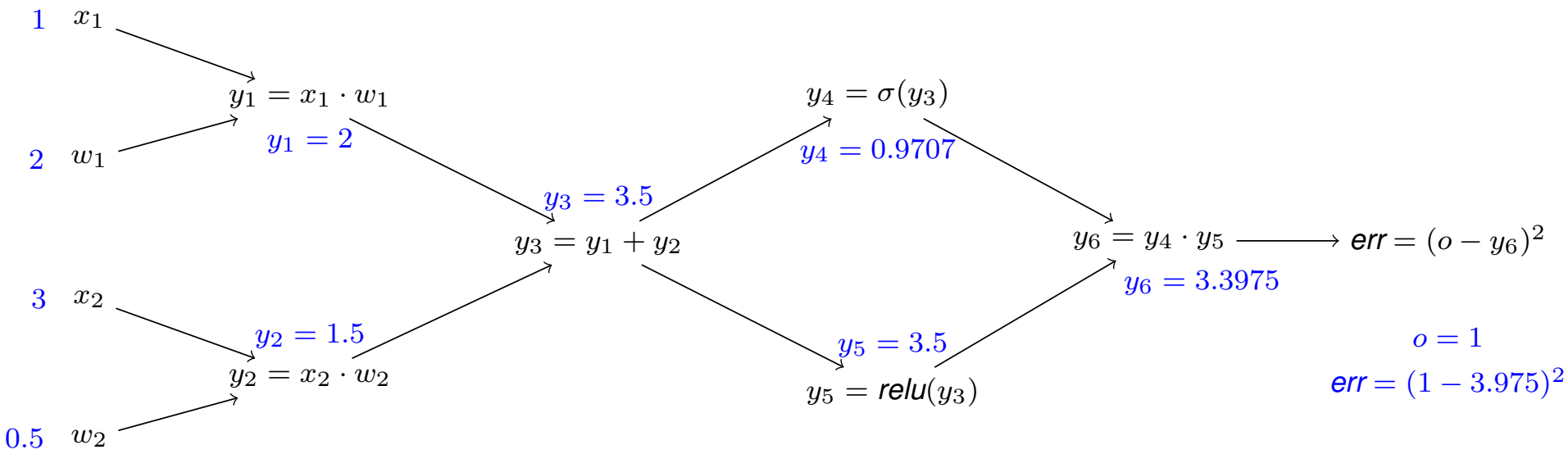


Forward pass

- Initialize and perform a full propagation through the network

Example

Computational graph

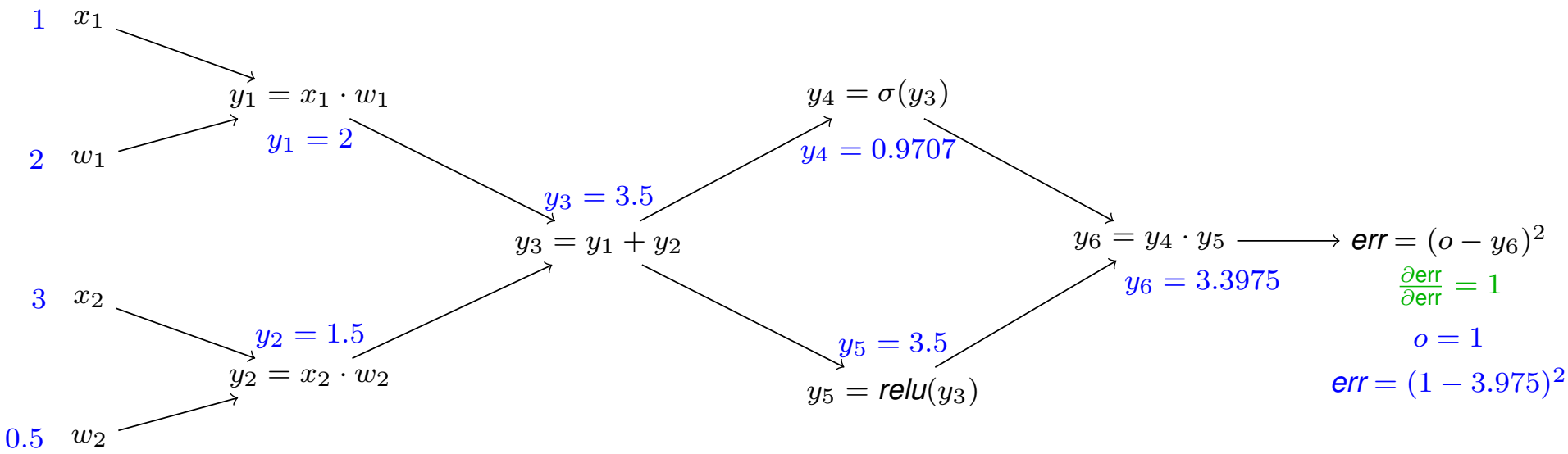


Forward pass

- Initialize and perform a full propagation through the network

Example

Computational graph



Back propagation Application of

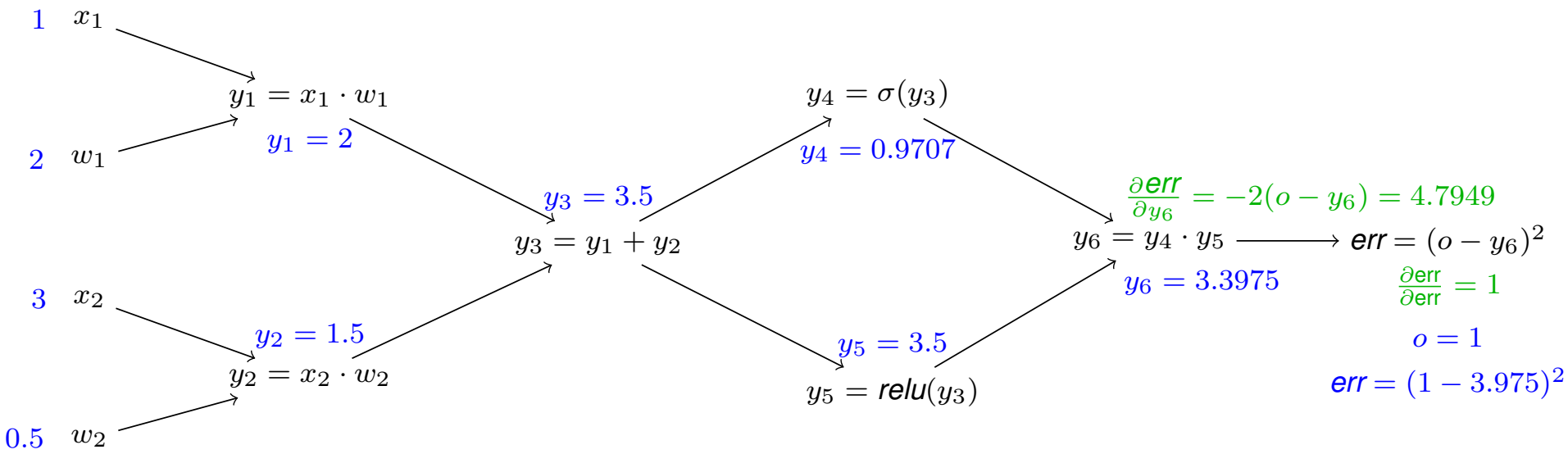
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

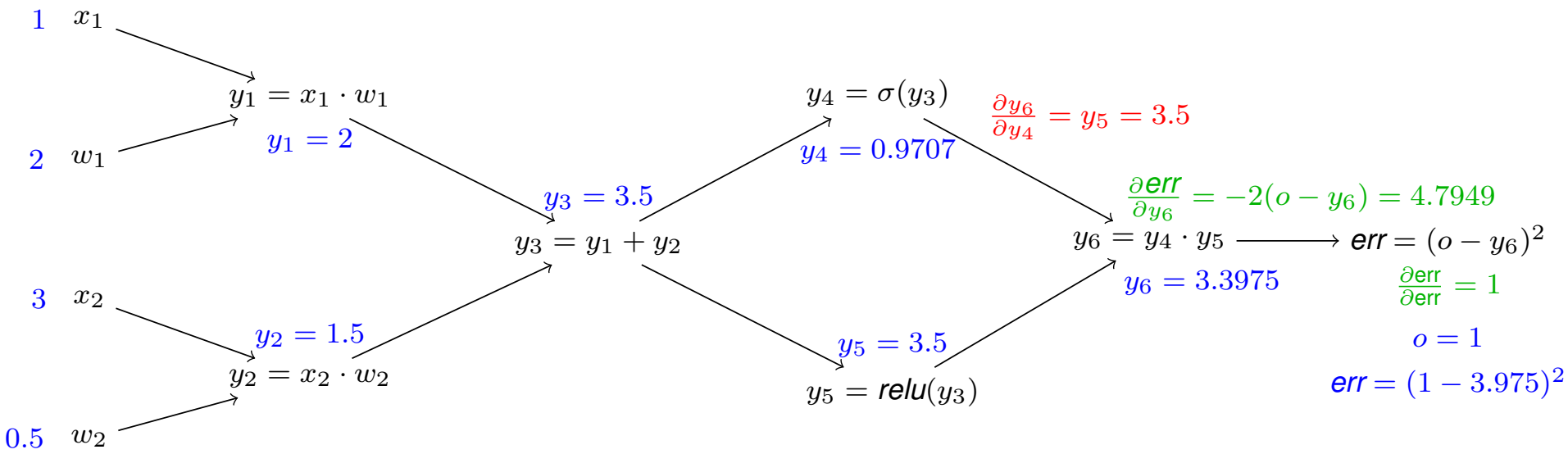
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

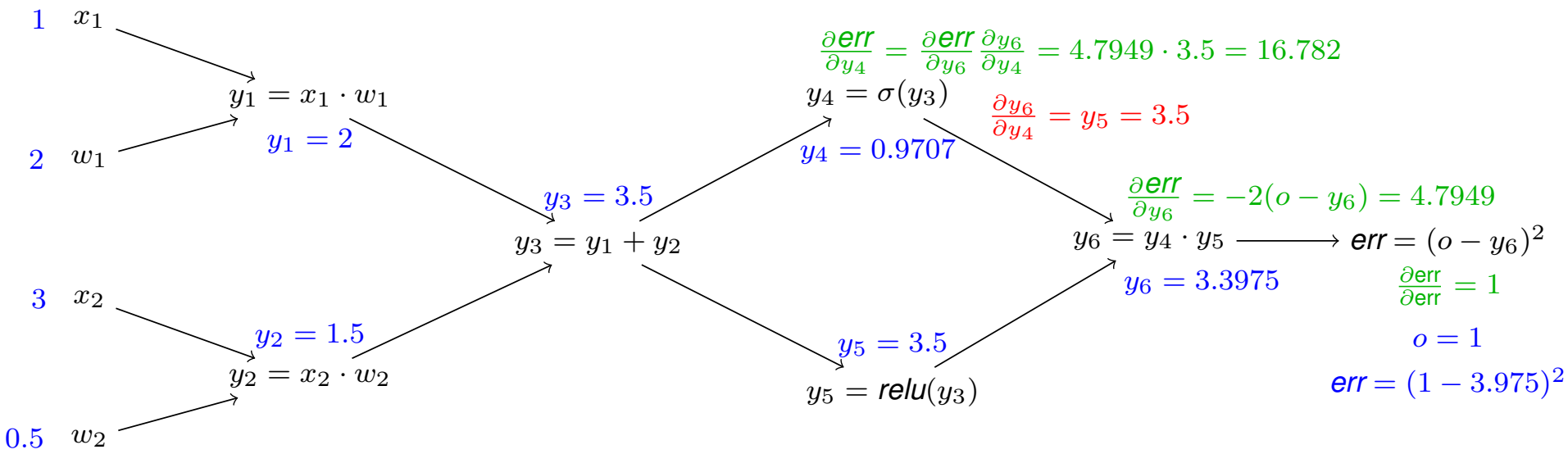
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

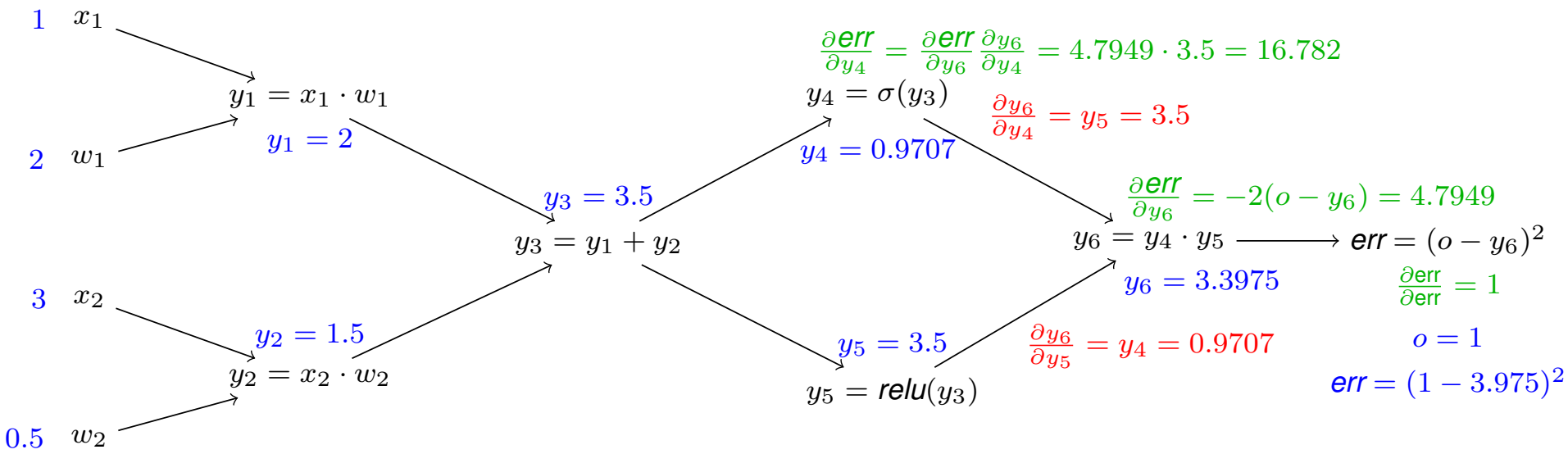
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

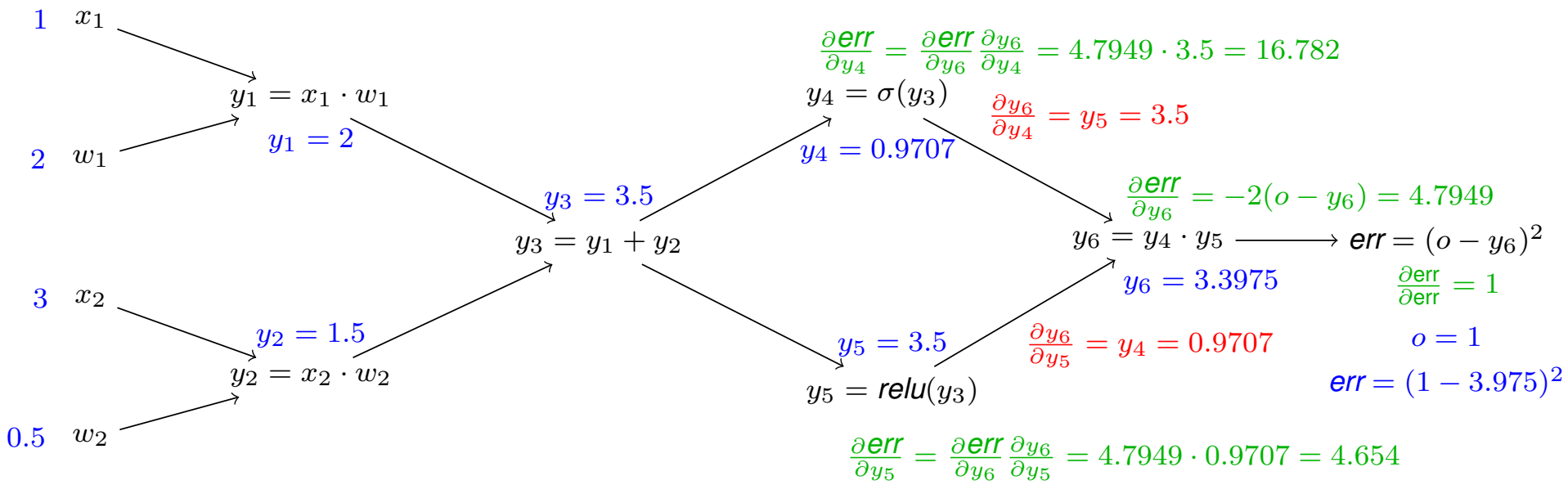
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

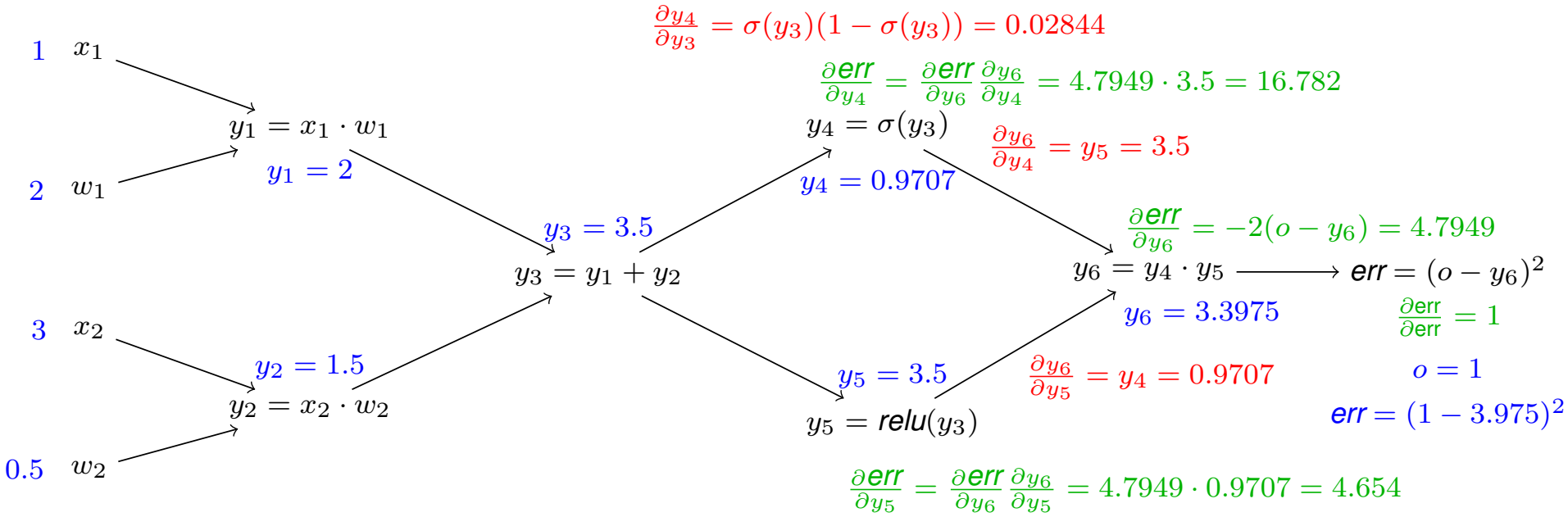
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

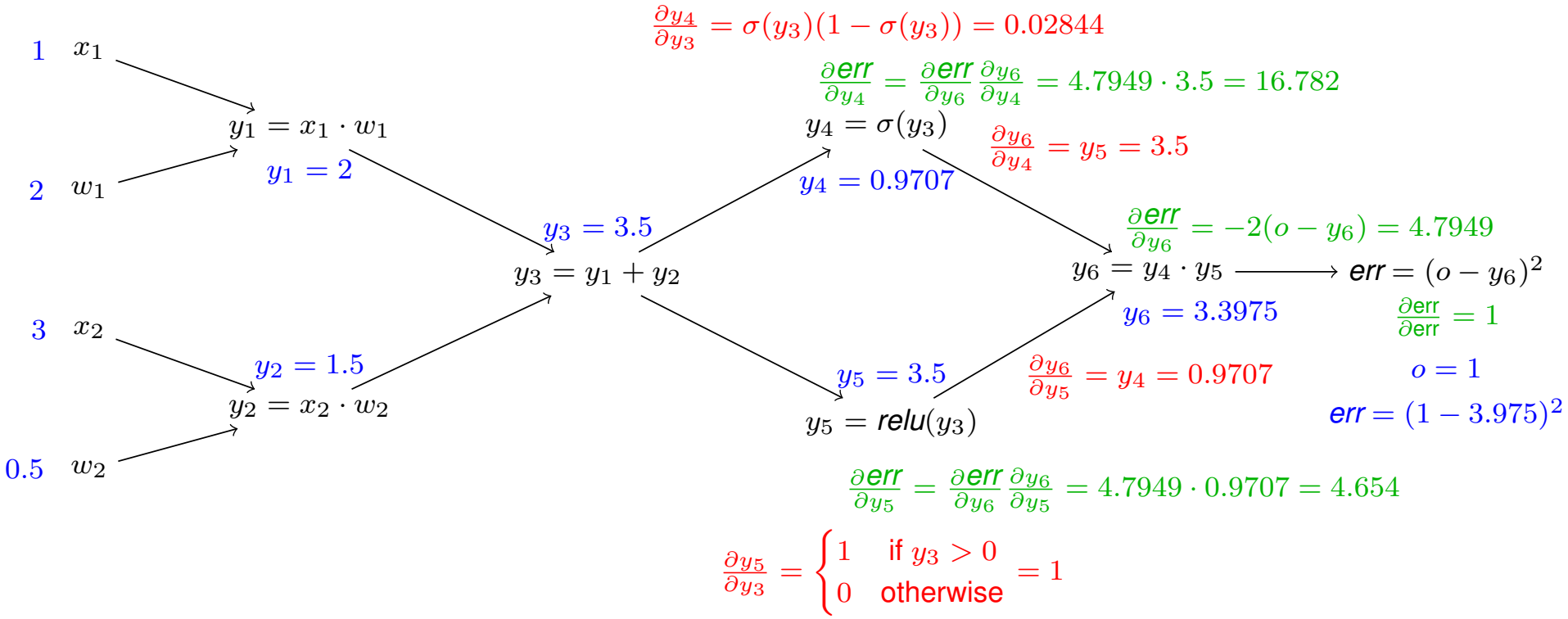
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

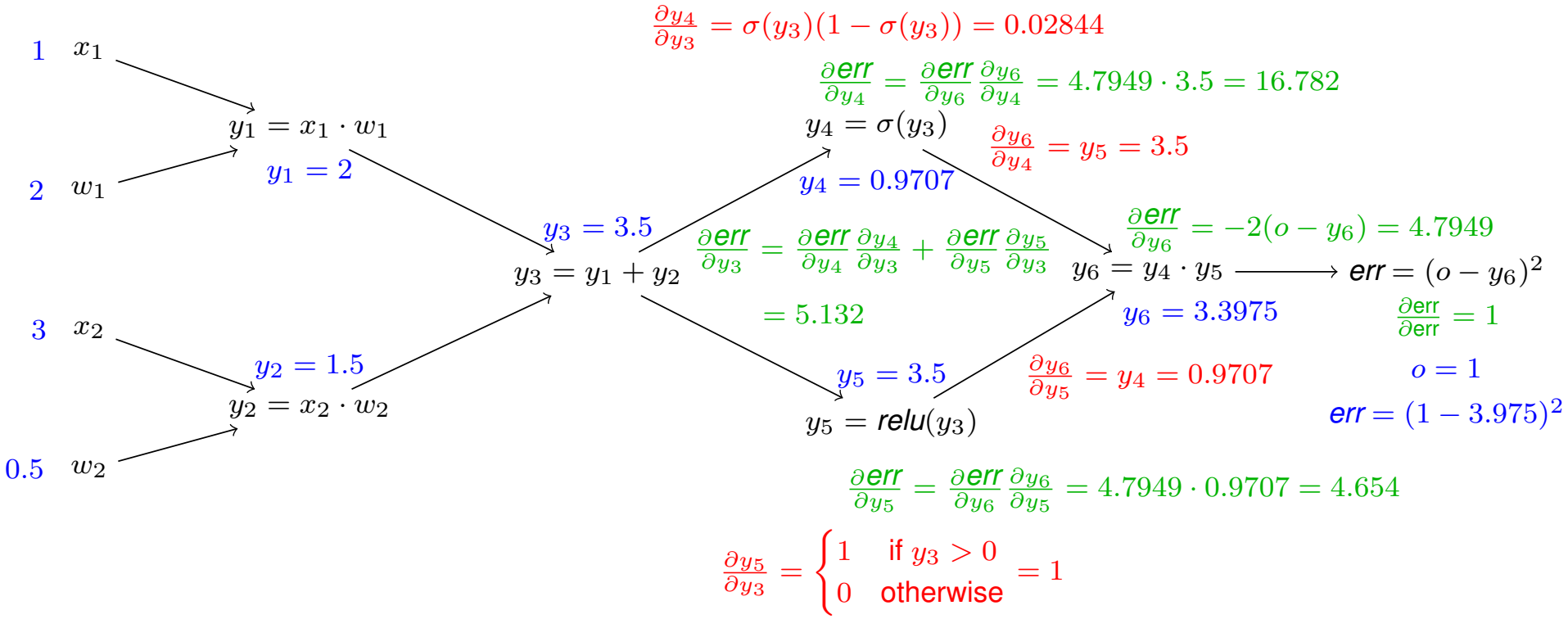
- Chain rule
- Dynamic programming

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

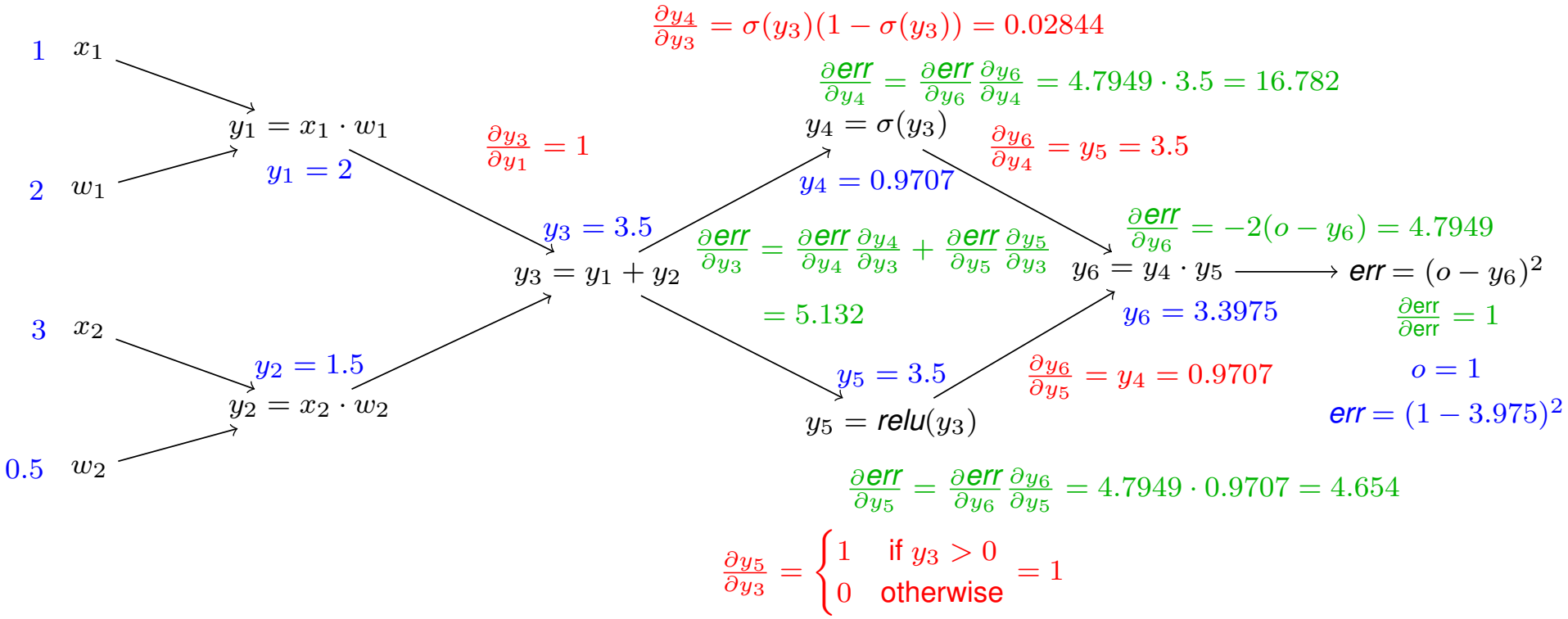
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

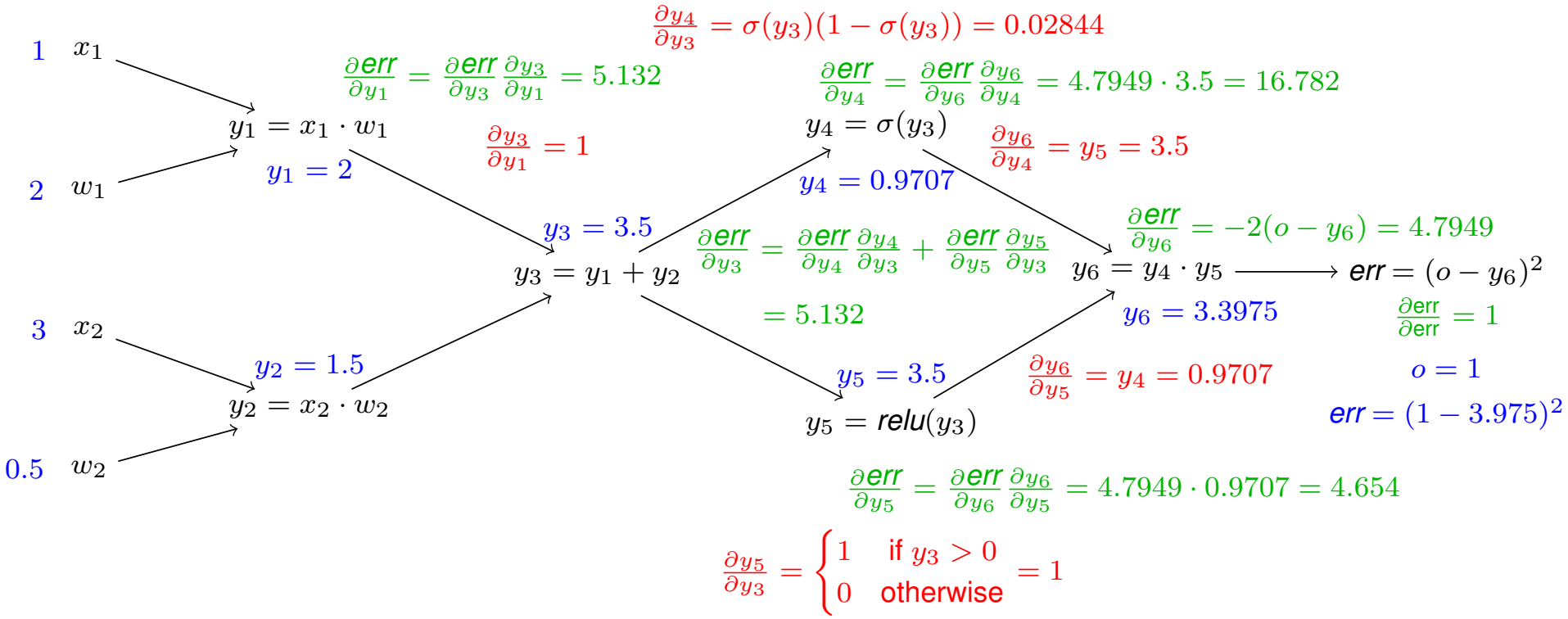
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

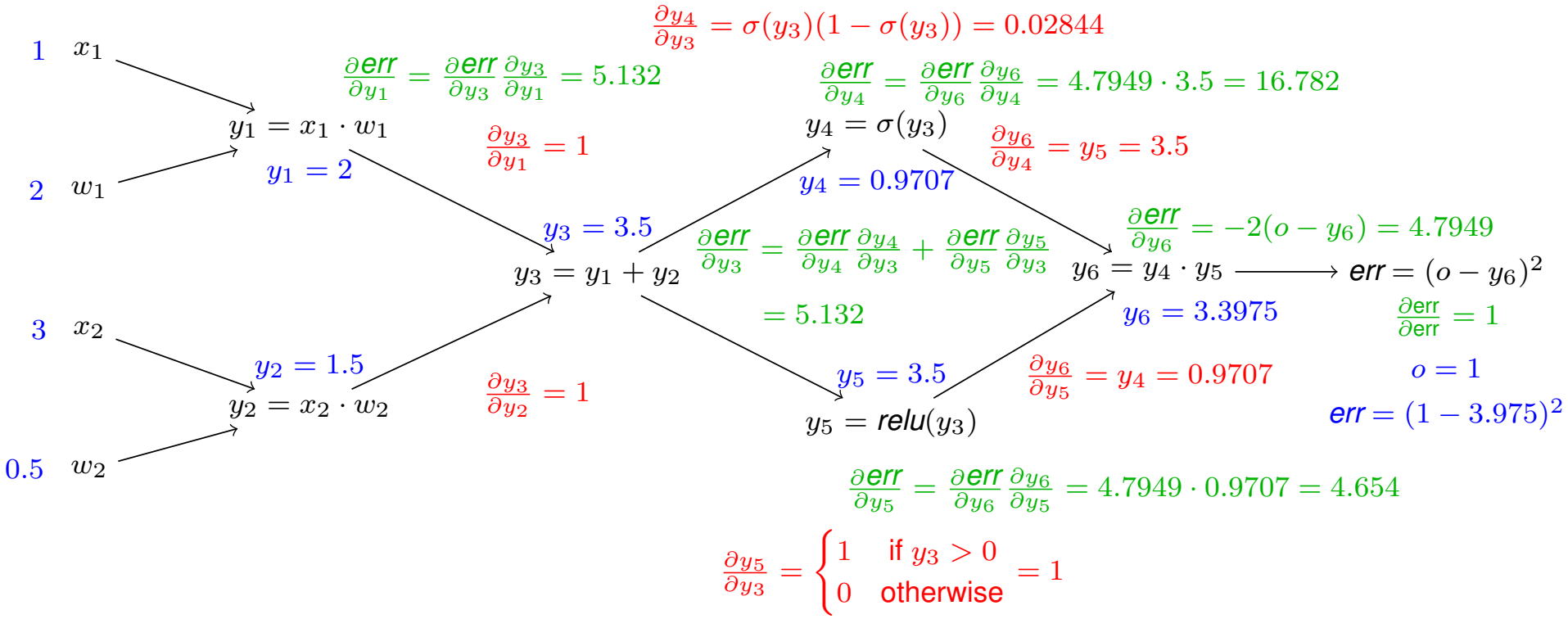
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

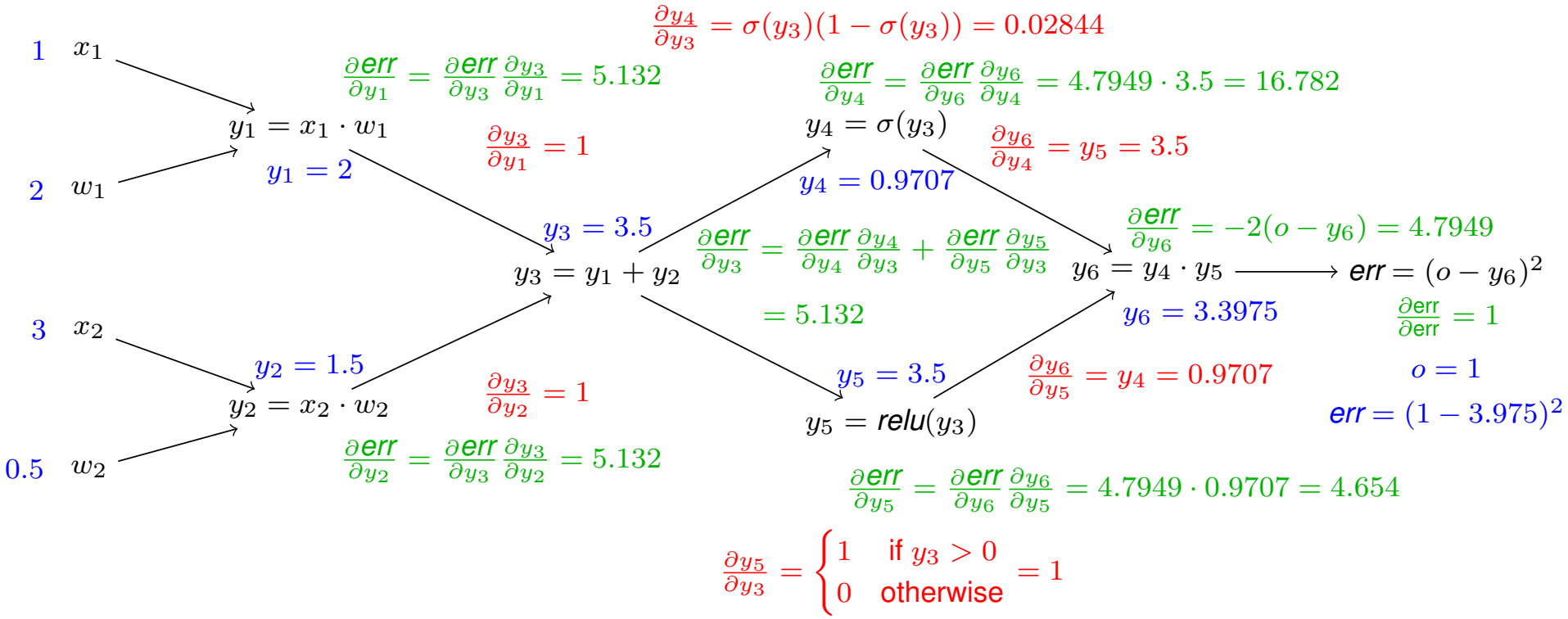
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

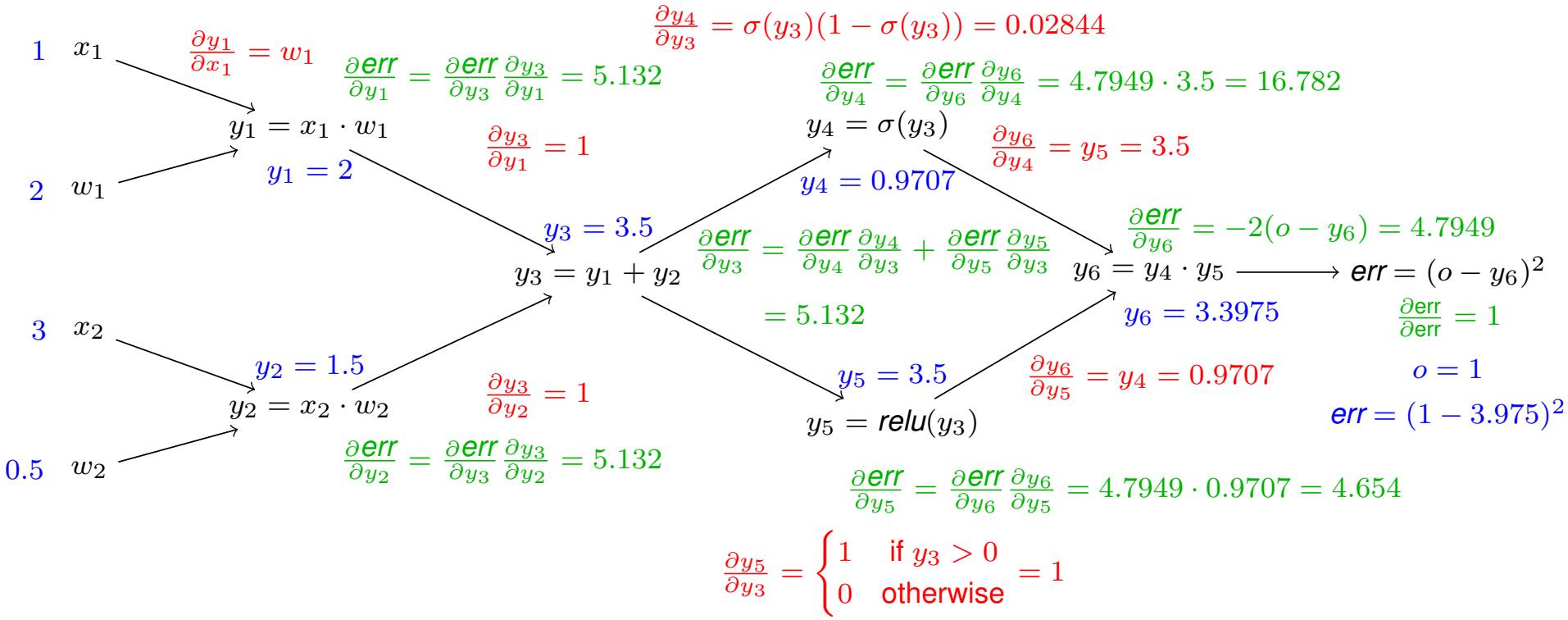
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

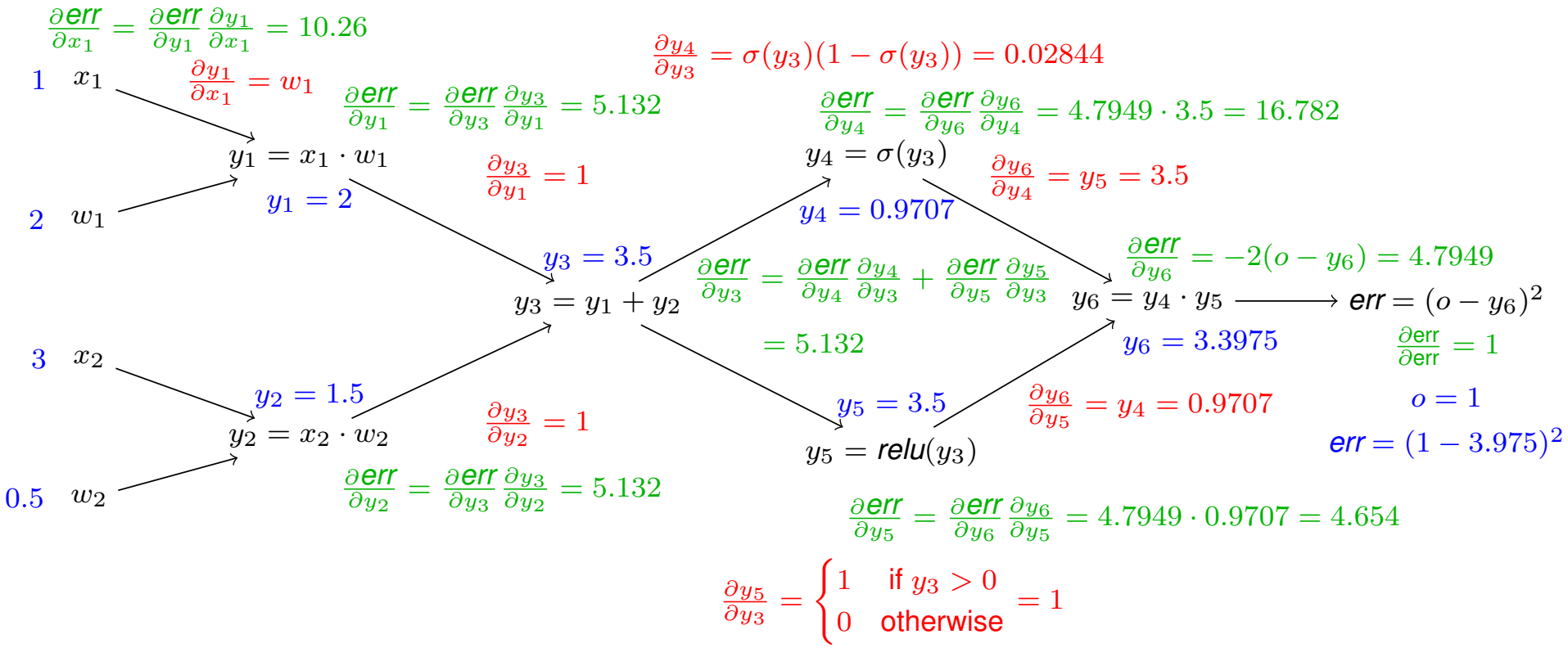
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

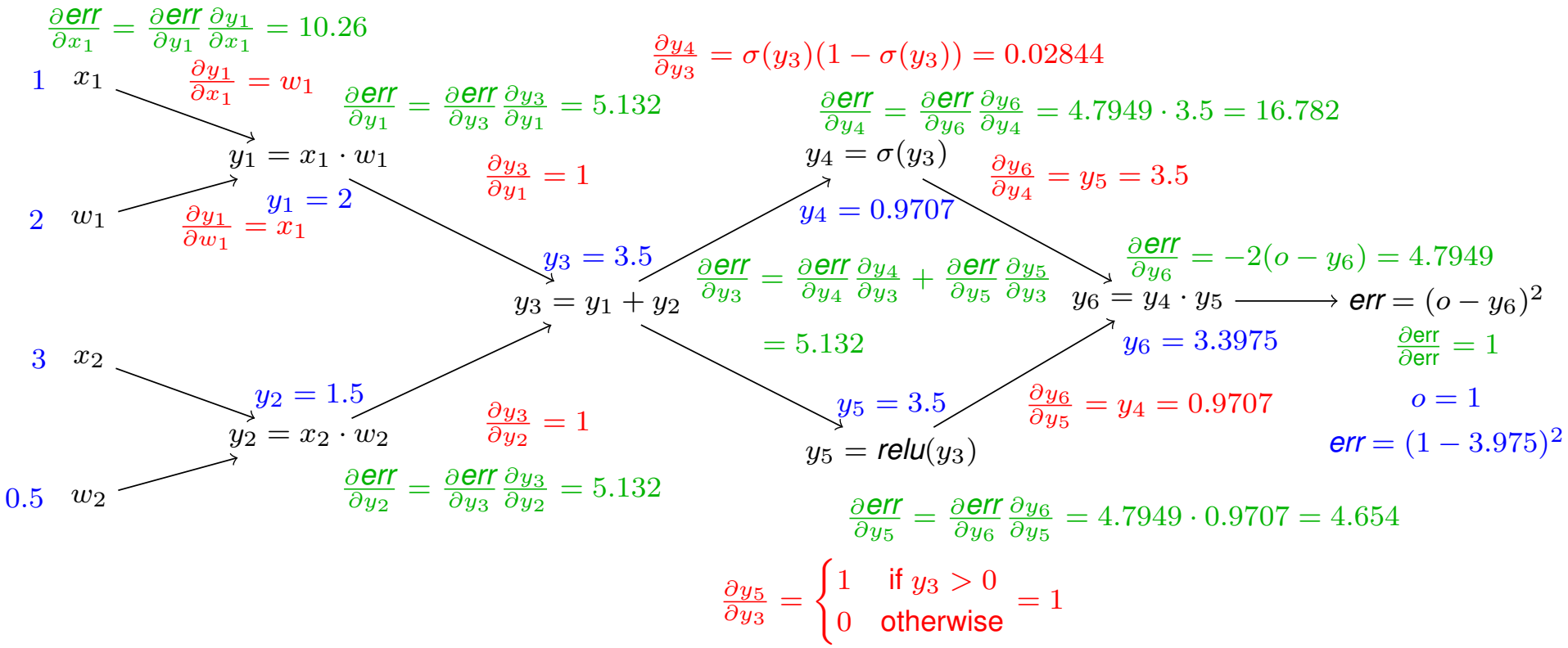
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

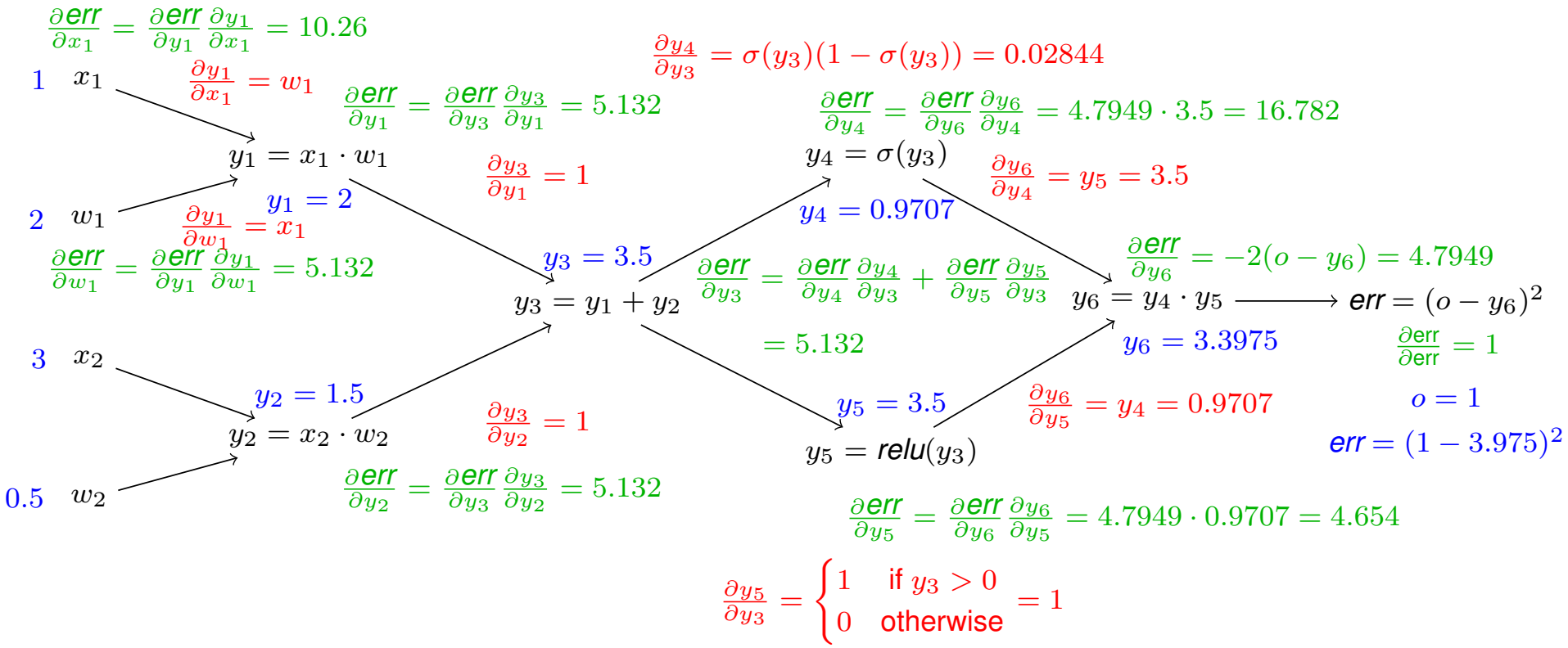
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

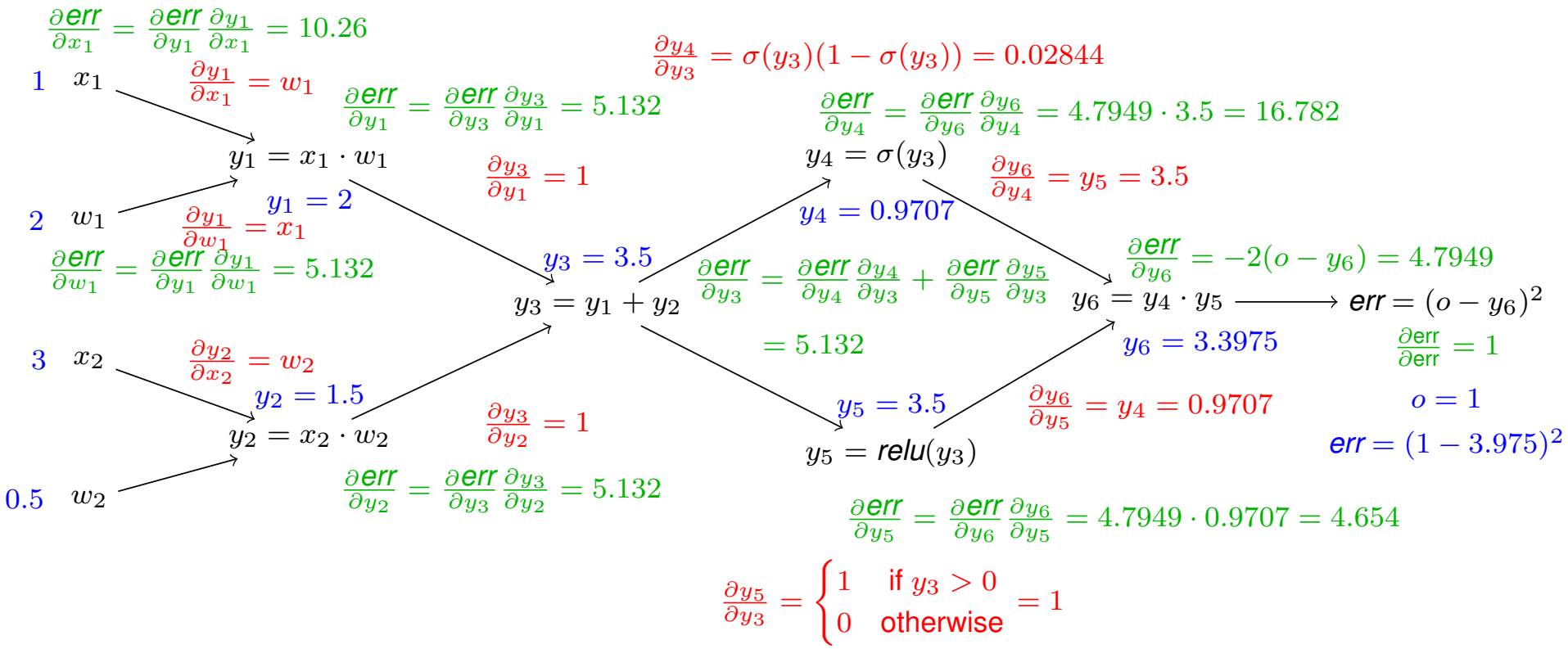
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

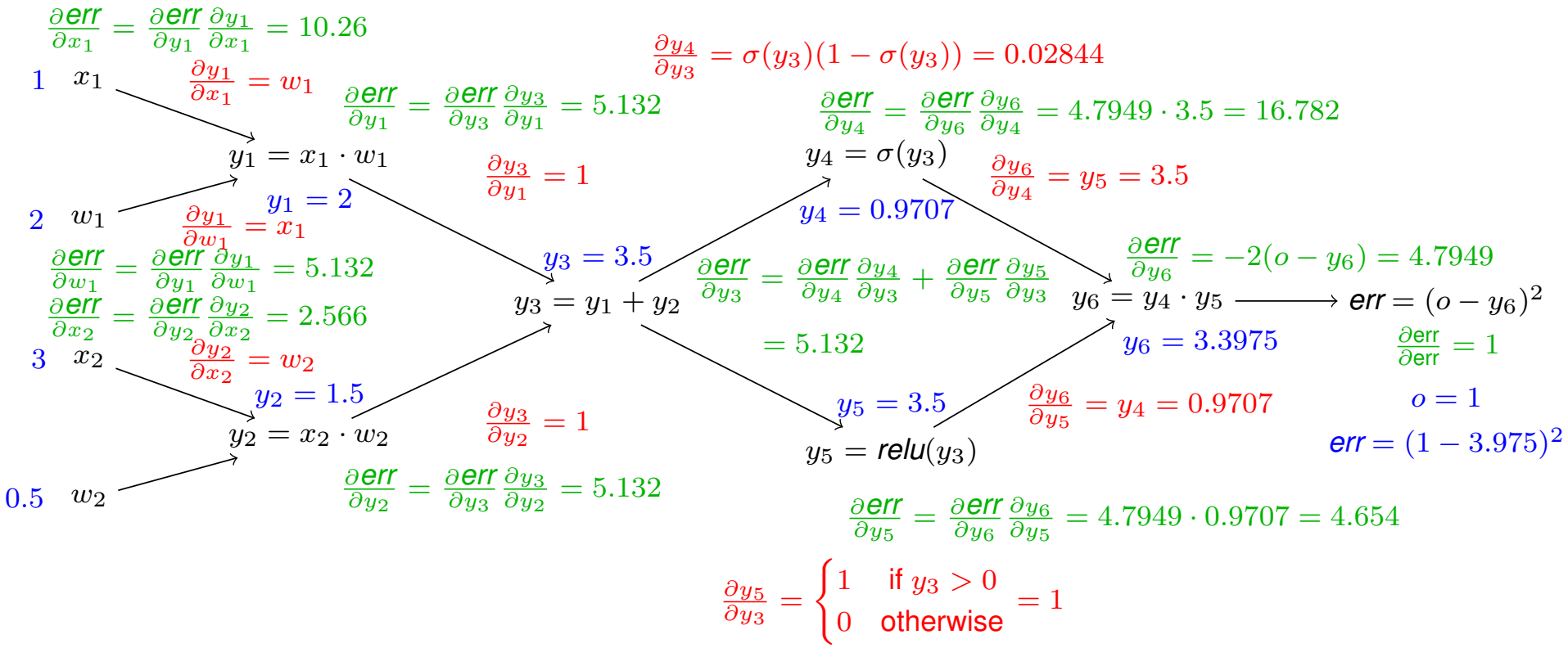
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

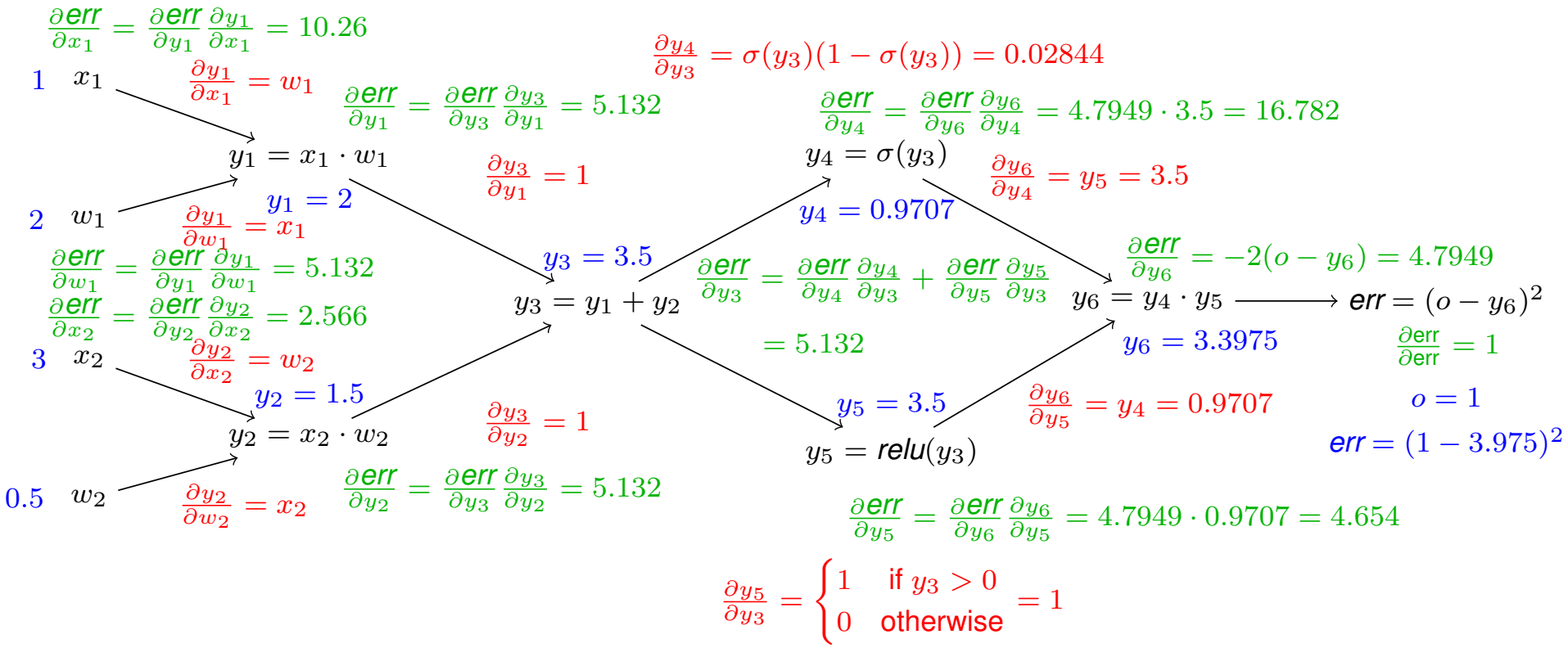
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

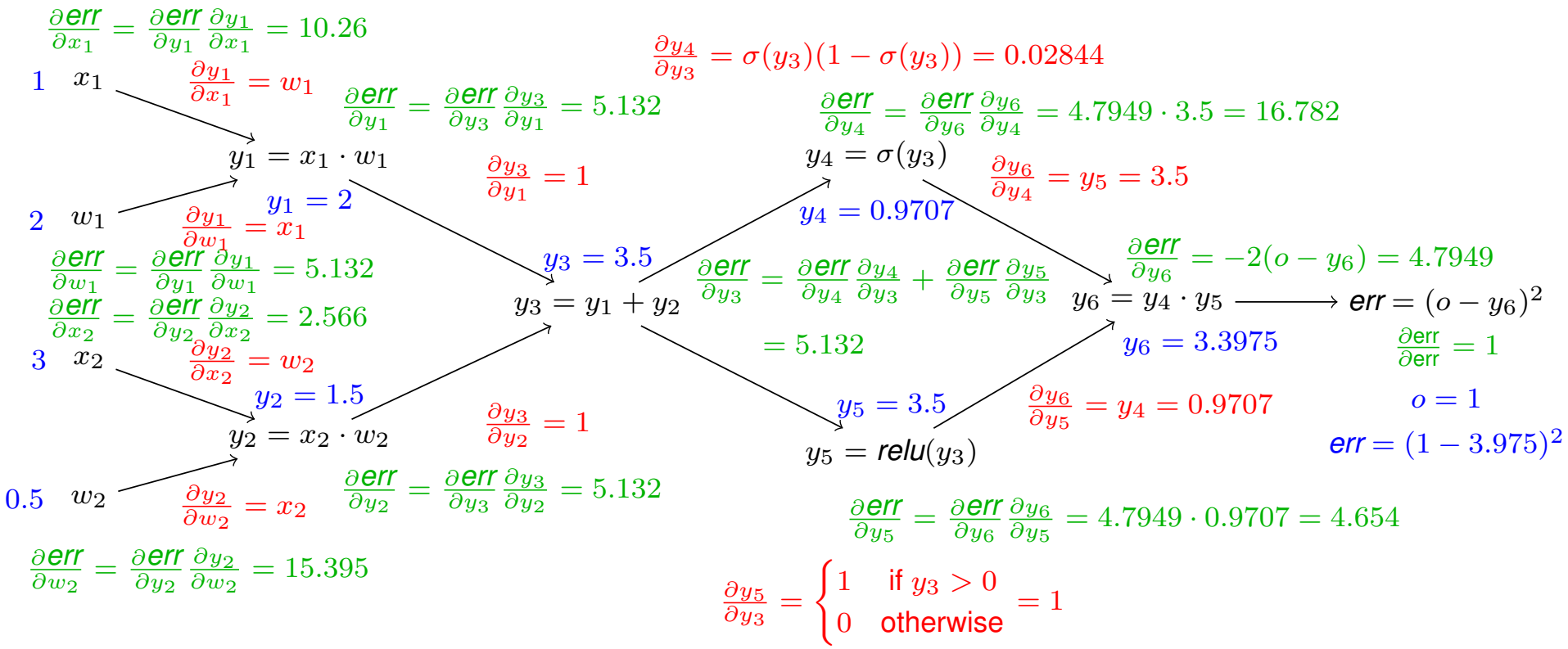
Note that sum at branching points

Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Example

Computational graph



Back propagation Application of

- Chain rule
- Dynamic programming

Note that sum at branching points

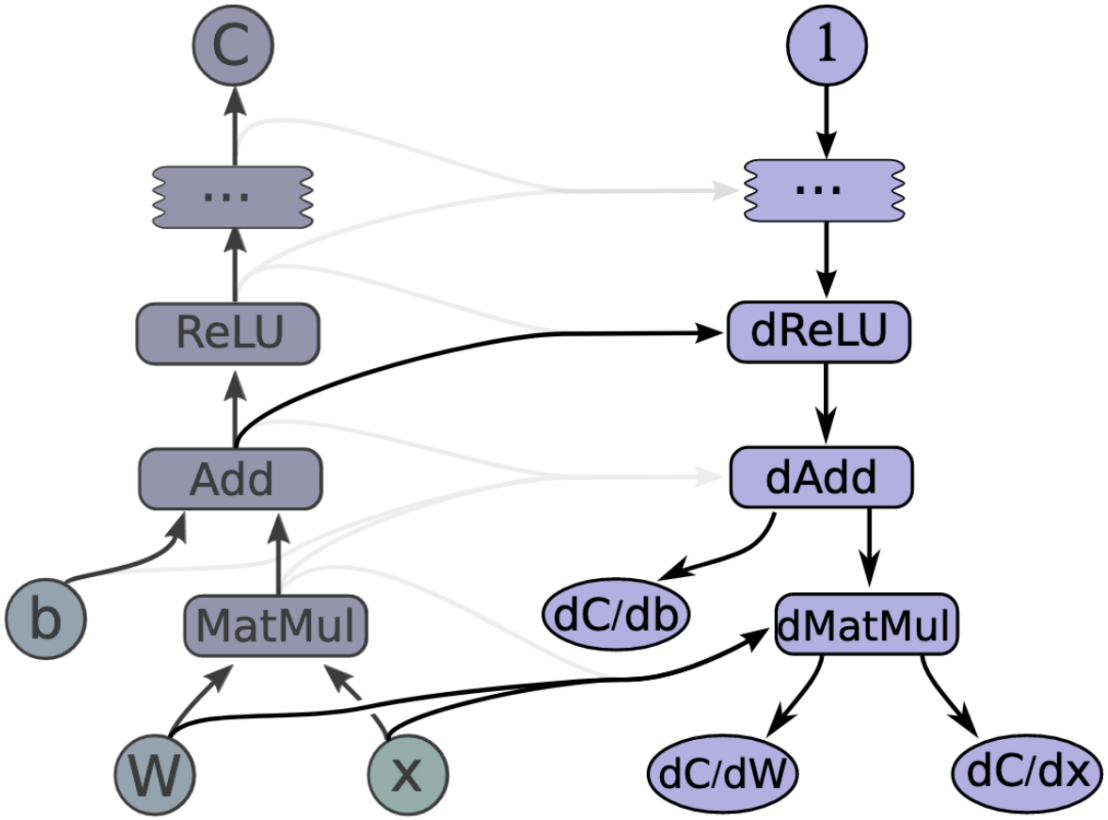
Recall chain rule

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)}\frac{\partial g(x)}{\partial x}$$

Back propagation and computational graphs

Implementing back propagation only requires the specification of

- the network structure
- the derivatives of the operations performed



Abadi, Martiín, et al. “Tensorflow: Large-scale machine learning on heterogeneous distributed systems.” (2016).

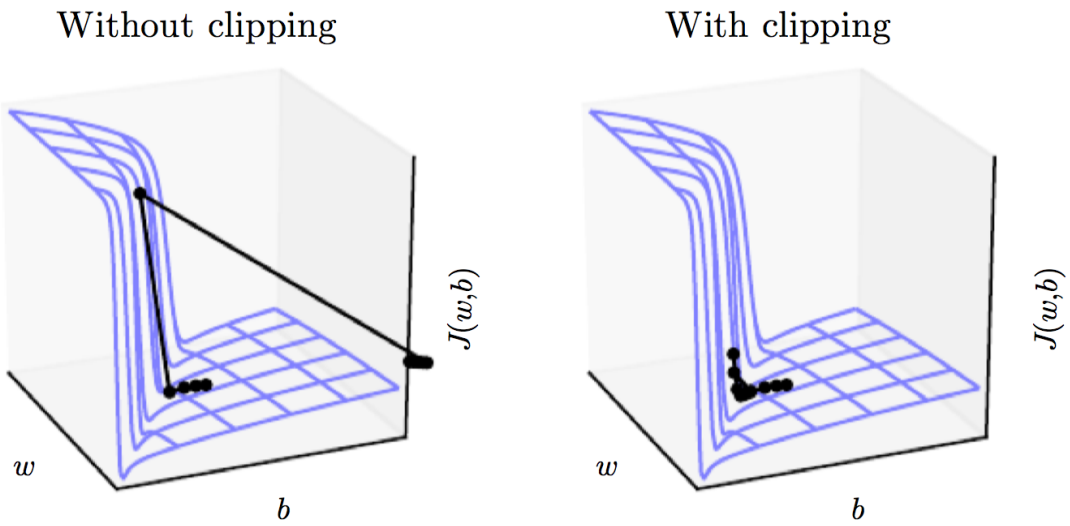
Jupyter notebook

gradient_example.py

More on derivatives and descent-based learning

Cliffs and exploding gradients

Neural networks with many layers can have steep regions in the cost landscape:



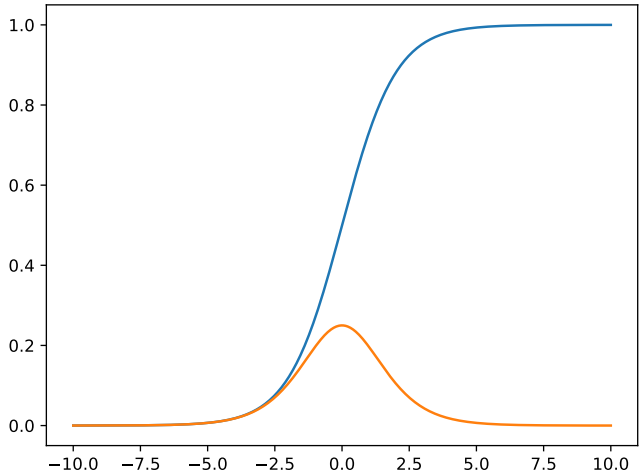
Idea

Clip the gradient before parameter updating:

$$\text{if } \|\mathbf{g}\| > v \text{ then } \mathbf{g} \leftarrow \frac{\mathbf{g}v}{\|\mathbf{g}\|}$$

Vanishing gradients

The sigmoid function and its derivative



If the weights are initialized too large:

- $\sigma(x)$ will be close to either 0 and 1
- $\frac{\partial}{\partial x}\sigma(x) = \sigma(x) \cdot (1 - \sigma(x))$ will be close to 0

$\rightsquigarrow$ gradients will vanish and the rest of the backward pass will be affected due to the multiplication in the chain rule.

Gradient-based learning

The cost function often decomposes over into a sum over the training examples

$$\mathit{Loss}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w})$$

For the log-loss: $\mathit{loss}(\mathbf{x}^{(i)}, \mathbf{y}^{(i)} \mid \mathbf{w}) = -\log p(y \mid \mathbf{x}, \mathbf{w})$.

Computing the gradient:

$$\nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \nabla_{\mathbf{w}} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w})$$

Gradient-based learning

The cost function often decomposes over into a sum over the training examples

$$\mathit{Loss}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w})$$

For the log-loss: $\mathit{loss}(\mathbf{x}^{(i)}, \mathbf{y}^{(i)} \mid \mathbf{w}) = -\log p(y \mid \mathbf{x}, \mathbf{w})$.

Computing the gradient:

$$\nabla_{\mathbf{w}} \mathit{Loss}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \nabla_{\mathbf{w}} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w})$$

Note:

- Computational cost is $O(m)$ $\rightsquigarrow$ increases with the sample size.
- The gradient is an (empirical) expectation.

Stochastic gradient descent

Idea: Approximate the expectation using a small batch of samples $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(B)}$ with B held fixed as sample size grows:

$$\mathbf{g} = \frac{1}{B} \sum_{i=1}^{B} \nabla_{\mathbf{w}} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w}).$$

Updating proceeds using the approximated gradient:

$$\mathbf{w}' = \mathbf{w} - \eta \mathbf{g}.$$

Terminology

- For a batch size of 1 the above approach is referred to as stochastic gradient descents.
- Otherwise, we have mini-batches and the method is referred to as mini-batch gradient descent.

Effect of increasing the batch size

- The standard error of the mean estimated from N examples is

$$SE(\hat{\mu}) = \sqrt{\text{Var}\left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)} = \frac{\sigma}{\sqrt{N}},$$

where σ is the true standard deviation of the samples.

Effect of increasing the batch size

- The standard error of the mean estimated from N examples is

$$SE(\hat{\mu}) = \sqrt{\text{Var}\left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)} = \frac{\sigma}{\sqrt{N}},$$

  where σ is the true standard deviation of the samples.
- $\rightsquigarrow$ less than linear return when using more samples:
  - Two estimate of the gradient: (a) based on 100 samples and (b) based on 10000 samples. (b) is 100 times slower, but reduces the standard error of the mean only by a factor 10.

Stochastic gradient descent: First observations

Effect of increasing the batch size

- The standard error of the mean estimated from N examples is

$$SE(\hat{\mu}) = \sqrt{\text{Var}\left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)} = \frac{\sigma}{\sqrt{N}},$$

  where σ is the true standard deviation of the samples.
- $\rightsquigarrow$ less than linear return when using more samples:
  - Two estimate of the gradient: (a) based on 100 samples and (b) based on 10000 samples. (b) is 100 times slower, but reduces the standard error of the mean only by a factor 10.

Getting an unbiased estimate

- The mini-batches must be selected randomly and should be independent of each other

Stochastic gradient descent: First observations

Effect of increasing the batch size

- The standard error of the mean estimated from N examples is

$$SE(\hat{\mu}) = \sqrt{\text{Var}\left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)} = \frac{\sigma}{\sqrt{N}},$$

  where σ is the true standard deviation of the samples.
- $\rightsquigarrow$ less than linear return when using more samples:
  - Two estimate of the gradient: (a) based on 100 samples and (b) based on 10000 samples. (b) is 100 times slower, but reduces the standard error of the mean only by a factor 10.

Getting an unbiased estimate

- The mini-batches must be selected randomly and should be independent of each other
- $\rightsquigarrow$ only achieved during the first pass of the data.

Stochastic gradient descent: First observations

Effect of increasing the batch size

- The standard error of the mean estimated from N examples is

$$SE(\hat{\mu}) = \sqrt{\text{Var}\left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)} = \frac{\sigma}{\sqrt{N}},$$

  where σ is the true standard deviation of the samples.
- $\rightsquigarrow$ less than linear return when using more samples:
  - Two estimate of the gradient: (a) based on 100 samples and (b) based on 10000 samples. (b) is 100 times slower, but reduces the standard error of the mean only by a factor 10.

Getting an unbiased estimate

- The mini-batches must be selected randomly and should be independent of each other
- $\rightsquigarrow$ only achieved during the first pass of the data.

Observation

$\rightsquigarrow$ SGD also suitable in a streaming setting.

Setting the learning rate

Because the stochastic gradient is noisy, the learning rate η should decrease over time to ensure convergence.

Sufficient conditions

$$\sum_{i=0}^{\infty} \eta_i = \infty \qquad \sum_{i=0}^{\infty} \eta_i^2 < \infty$$

In practice

- Adjust the learning rate during training by, e.g., reducing the learning rate according to a predefined schedule.
- May use different learning rates for different parameters.

Setting the learning rate

Because the stochastic gradient is noisy, the learning rate η should decrease over time to ensure convergence.

Sufficient conditions

$$\sum_{i=0}^{\infty} \eta_i = \infty \qquad \sum_{i=0}^{\infty} \eta_i^2 < \infty$$

In practice

- Adjust the learning rate during training by, e.g., reducing the learning rate according to a predefined schedule.
- May use different learning rates for different parameters.

Alternatively: Adaptive learning rates (AdaGrad, Adam, . . .).

Note:

- Different learning rates can be used for different parameters (AdaGrad, . . .).
- For an overview, see `http://ruder.io/optimizing-gradient-descent/`

Momentum

Designed to accelerate learning when faced with high curvature, small or noisy gradients.

Idea

Introduce a variable $\mathbf{v}$ (velocity/momentum) to determine the direction of parameter movement:

$$\mathbf{v} \leftarrow \alpha\mathbf{v} - \eta\nabla_{\mathbf{w}} \left(\frac{1}{N} \sum_{i=1}^{N} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w}) \right)$$

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$$

- $\mathbf{v}$ - an exponentially decaying average of the negative gradient

Momentum

Designed to accelerate learning when faced with high curvature, small or noisy gradients.

Idea

Introduce a variable $\mathbf{v}$ (velocity/momentum) to determine the direction of parameter movement:

$$\mathbf{v} \leftarrow \alpha\mathbf{v} - \eta\nabla_{\mathbf{w}} \left(\frac{1}{N} \sum_{i=1}^{N} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w}) \right)$$

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$$

- $\mathbf{v}$ - an exponentially decaying average of the negative gradient

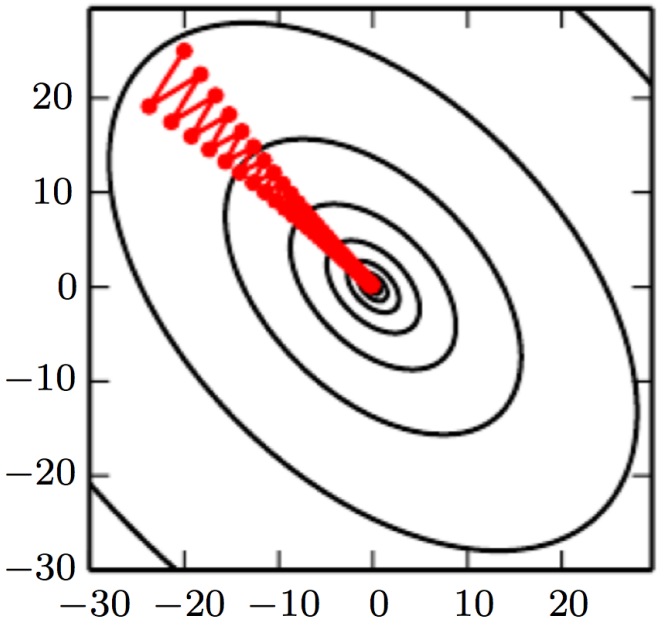
Note: Maximum speed when gradients point in the same direction $-\mathbf{g}$ giving terminal velocity

$$\frac{\eta\|\mathbf{g}\|}{1-\alpha}$$

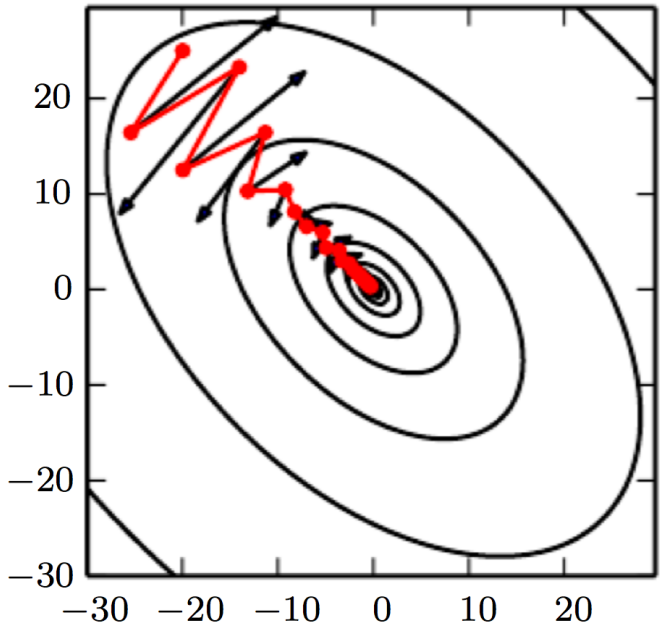
Think in terms of $1/(1-\alpha)$: $\alpha = 0.9 \rightsquigarrow$ multiply max speed by 10 rel. to GDS.

Effect of momentum

Illustration



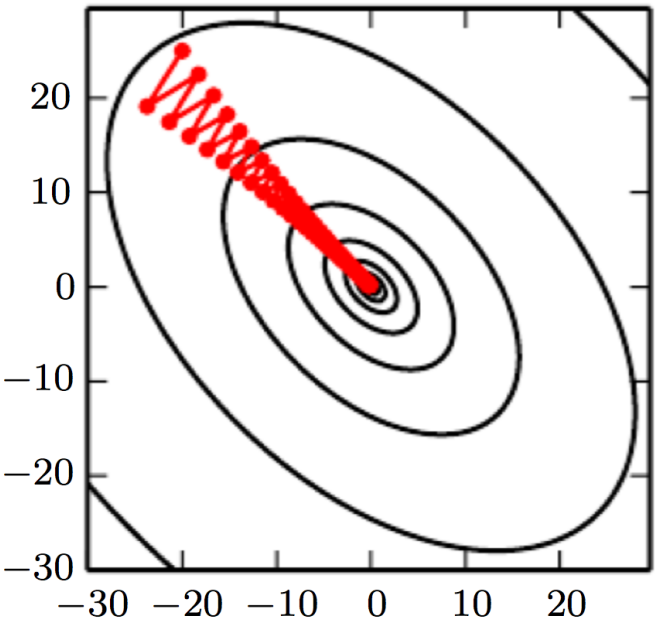
Without momentum



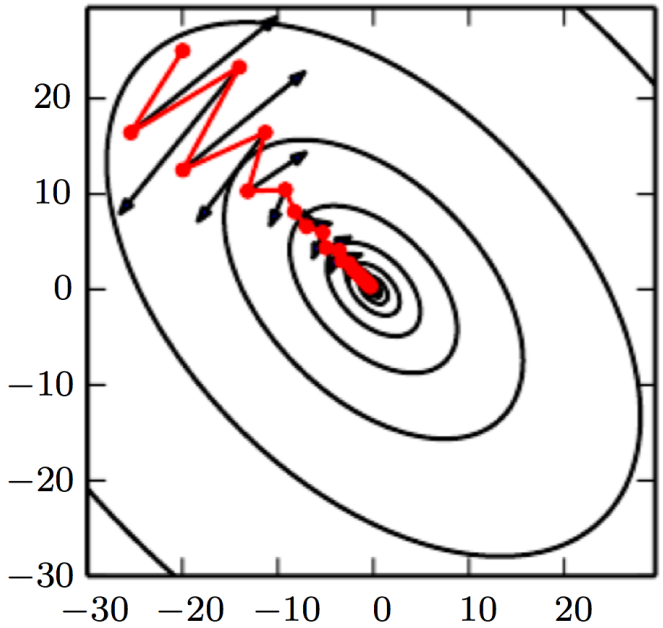
With momentum

Effect of momentum

Illustration



Without momentum



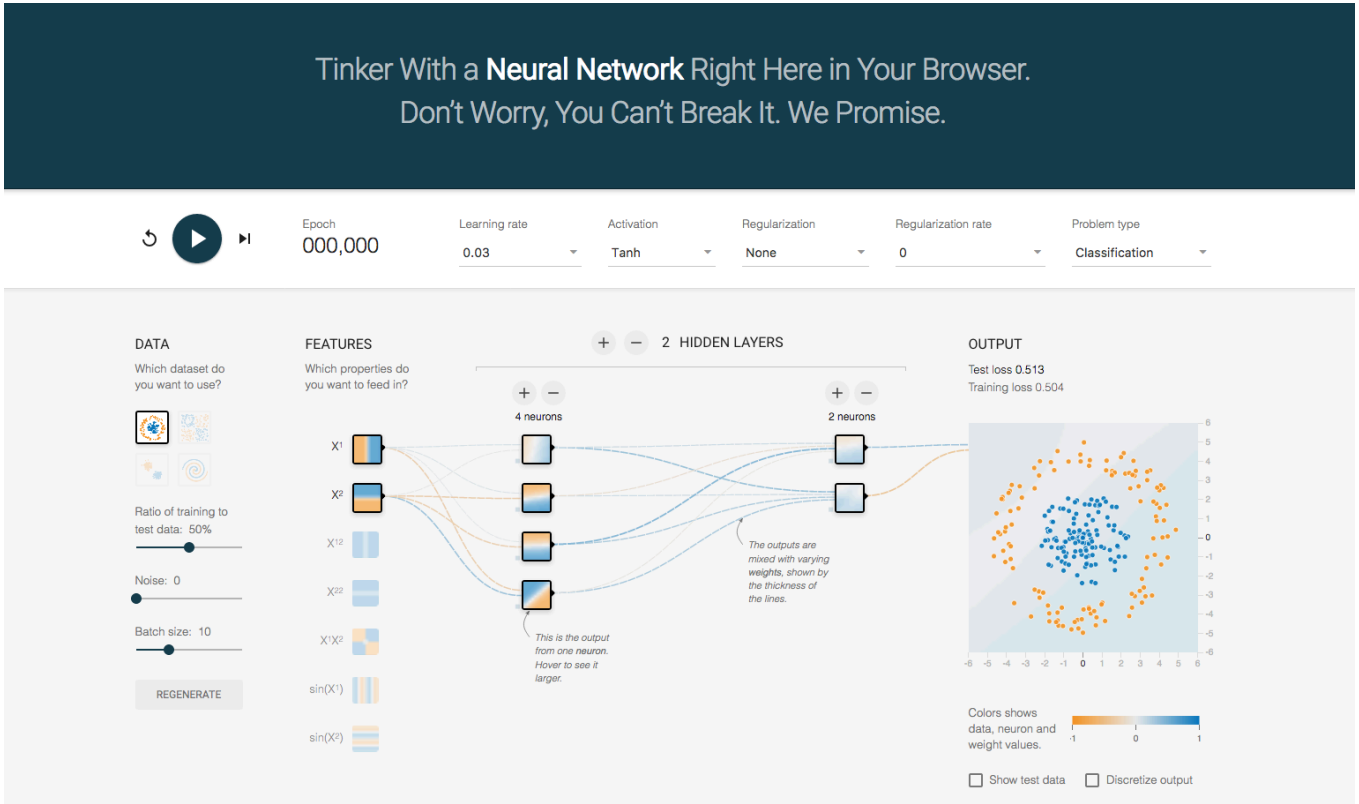
With momentum

Momentum variants: Nesterov momentum

Gradient is evaluated after current velocity has been applied:

$$\mathbf{v} \leftarrow \alpha\mathbf{v} - \eta\nabla_{\mathbf{w}}\left(\frac{1}{N}\sum_{i=1}^{N} \mathit{loss}(\mathbf{x}^{(i)}, y^{(i)} \mid \mathbf{w} + \alpha\mathbf{v})\right)$$

Playground



playground.tensorflow.org